



**POLSKO-JAPOŃSKA AKADEMIA
TECHNIK KOMPUTEROWYCH**

WYDZIAŁ INFORMATYKI

KATEDRA MECHANIKI, ROBOTYKI I INFORMATYKI

ROBOTYKA

FILIP PAŃCZAK

Nr albumu: s25056

**Implementacja systemu sterowania i stabilizacji lotu drona
quadrotor z regulatorem PID**

Praca inżynierska

Mgr. Inż. Maciej Maciejewski

Warszawa, luty, 2026

Streszczenie

W niniejszej pracy przedstawiono projekt oraz implementację systemu sterowania i stabilizacji lotu drona typu quadrotor z wykorzystaniem regulatora PID. Drony wielowirnikowe należą do układów z natury niestabilnych, dlatego ich poprawne sterowanie wymaga ciągłego przetwarzania danych z czujników inercyjnych oraz szybkiej reakcji układu sterującego. Celem pracy było zaprojektowanie i wykonanie systemu, który umożliwia stabilny lot drona sterowanego ręcznie, przy jednoczesnym kompensowaniu zakłóceń i odchylen od zadanej orientacji.

W pracy omówiono podstawy stabilizacji lotu drona quadrotor, zasadę działania regulatora PID oraz rolę czujników inercyjnych w systemie sterowania. Następnie zaprezentowano architekturę zaprojektowanego systemu oraz szczegóły implementacji sprzętowej i programowej z wykorzystaniem mikrokontrolera STM32. Zaimplementowany regulator PID odpowiada za stabilizację kątów orientacji drona, a jego parametry zostały dobrane metodą prób i błędów.

Przeprowadzone testy potwierdziły poprawne działanie systemu oraz możliwość stabilnego sterowania dronem w warunkach rzeczywistych. Otrzymane wyniki pokazują, że zastosowanie regulatora PID pozwala skutecznie poprawić stabilność lotu drona quadrotor.

Słowa kluczowe: dron quadrotor, regulator PID, stabilizacja lotu, STM32, system sterowania



**POLISH-JAPANESE ACADEMY
OF INFORMATION TECHNOLOGY**

FACULTY OF COMPUTER SCIENCE

DEPARTMENT OF MECHANICS, ROBOTICS AND COMPUTER SCIENCE

ROBOTICS

FILIP PAŃCZAK

ID: s25056

**Implementation of a Control and Flight Stabilization System for a
Quadrotor Drone Using a PID Controller**

Engineering thesis

Mgr. Inż. Maciej Maciejewski

Warsaw, February, 2026

Abstract:

This thesis presents the design and implementation of a control and flight stabilization system for a quadrotor drone using a PID controller. Quadrotor drones are inherently unstable systems, therefore maintaining stable flight requires continuous processing of data from inertial sensors and rapid response of the control system. The main objective of this work was to develop a system that enables stable manual control of a drone while compensating for disturbances and deviations from the desired orientation.

The thesis discusses the basic principles of quadrotor flight stabilization, the operation of a PID controller, and the role of inertial sensors in control systems. Next, the architecture of the proposed system is presented, along with details of the hardware and software implementation based on an STM32 microcontroller. The implemented PID controller is responsible for stabilizing the drone's orientation angles, and its parameters were tuned using a trial-and-error method.

The conducted tests confirmed the correct operation of the system and demonstrated the possibility of achieving stable flight under real operating conditions. The results show that the use of a PID controller significantly improves the flight stability of a quadrotor drone.

Keywords: quadrotor drone, PID controller, flight stabilization, STM32, control system

SPIS TREŚCI

1. WSTĘP	8
1.1. Wprowadzenie	8
1.2. Cel pracy	9
1.3. Zakres pracy	10
2. STABILIZACJA LOTU DRONA QUADROTOR.....	10
2.1. Dron quadrotor	10
2.2. Czujniki inercyjne w stabilizacji lotu	12
2.2.1. Akcelerometr	12
2.2.2. Żyroskop	13
2.3. Pojęcie stabilizacji lotu	13
3. REGULATOR PID W SYSTEMIE STEROWANIA DRONEM.....	13
3.1. Zasada działania regulatora PID.....	14
3.2. Człony regulatora PID.....	15
3.2.1. Człon proporcjonalny (P)	15
3.2.2. Człon całkujący (I)	15
3.2.3. Człon różniczkujący (D)	15
3.3. Zastosowanie regulatora PID w stabilizacji drona	16
3.4. Strojenie regulatora PID	16
4. PROJEKT SYSTEMU STEROWANIA DRONA	16
4.1. Założenia projektowe	17
4.2. Architektura systemu sterowania	17
4.3. Struktura algorytmu sterowania	18
4.4. Projekt elektroniki sterującej	18
4.5. Wykorzystane narzędzia programowe	18
5. IMPLEMENTACJA SYSTEMU STEROWANIA	19
5.1. Implementacja sprzętowa	19
5.1.1. Dedykowana płytka PCB sterownika lotu	19

5.1.2 Zasilanie układu	22
5.1.3. Mikrokontroler STM32L432KC	22
5.1.4. Czujnik inercyjny IMU BNO055.....	23
5.1.5. Odbiornik RC FS-IA10B.....	23
5.1.6. Regulatory ESC BLHeli_S 20A.....	23
5.1.7. Silniki bezszczotkowe (BLDC) BR2205 2300KV	24
5.2. Implementacja programowa	24
5.2.1. Biblioteka HAL (Hardware Abstraction Layer)	24
5.2.1.1. Struktura i sposób działania biblioteki HAL	25
5.2.1.2. Zastosowanie HAL w projekcie dron	25
5.2.1.3. Zalety wykorzystania biblioteki HAL	26
5.2.2. Środowisko programistyczne i konfiguracja systemu.....	26
5.2.3. Struktura programu	27
5.2.4. Synchronizacja pętli sterowania	28
5.2.5. Odczyt danych z czujnika inercyjnego.....	29
5.2.6. Odbiór i przetwarzanie sygnałów sterowania RC	30
5.2.6.1. Odbiornik RC i protokół iBUS	30
5.2.6.2. Odbiór danych UART z wykorzystaniem DMA.....	30
5.2.6.3. Dekodowanie ramek iBUS	30
5.2.6.4. Mapowanie kanałów sterujących.....	31
5.2.6.5. Zadawanie wartości sterujących z pilota.....	32
5.2.7. Implementacja regulatora PID	32
5.2.8. Wyznaczanie pomiarów orientacji.....	34
5.2.9. Warstwa bezpieczeństwa i reset regulatorów.....	34
5.2.10. Procedura kalibracji regulatorów ESC	35
5.2.11. Obliczenia regulatorów PID i miksowanie sygnałów	36
5.2.12. Warunek minimalnego położenia gazu.....	37
6. TESTY DZIAŁANIA SYSTEMU	37
6.1. Stanowisko testowe	37
6.1.2. Druga wersja stanowiska.....	39
6.2. Metodyka przeprowadzania testów	39
6.3. Strojenie regulatora PID z wykorzystaniem zmiennych modyfikowanych w debuggerze	39
6.4. Obserwacje zachowania systemu	40
6.4.1. Wpływ poszczególnych członów regulatora na zachowanie drona	42
6.4.1.1. Zachowanie układu przy zbyt dużym wzmacnieniu Kp	43
6.4.1.2. Wpływ członu całkującego Ki	44
6.4.1.3 Wpływ członu różniczkującego Kd.....	44
6.4.2. Działanie mechanizmów bezpieczeństwa	46
6.3. Ograniczenia przeprowadzonych testów	46
7. PODSUMOWANIE I WNIOSKI	46

7.1. Cel i zakres wykonanej pracy	46
7.2. Ocena działania systemu	46
7.3. Wnioski z testów i strojenia regulatora PID.....	47
7.4. Ograniczenia rozwiązania	48
7.5. Możliwości dalszego rozwoju	48
BIBLIOGRAFIA	50
SPIS RYSUNKÓW.....	51

1. Wstęp

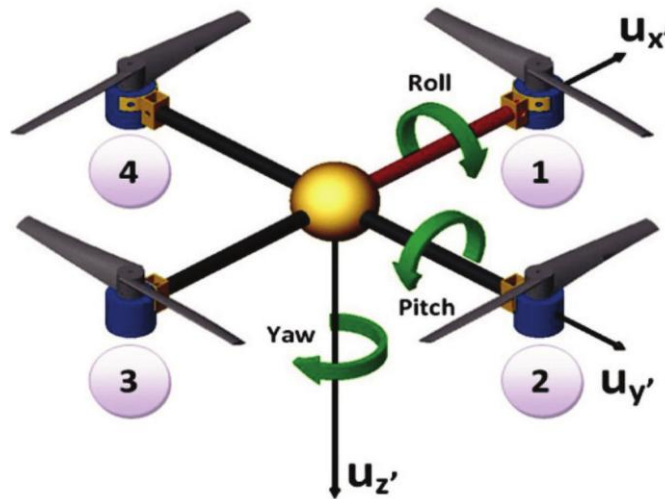
Bezzałogowe statki powietrzne, potocznie nazywane dronami, znajdują obecnie szerokie zastosowanie zarówno w zastosowaniach komercyjnych, jak i badawczych. (1) Szczególną popularność zyskały drony wielowirnikowe, w tym konstrukcje typu quadrotor, charakteryzujące się prostą budową mechaniczną oraz dużą zwrotnością. Pomimo stosunkowo nieskomplikowanej konstrukcji mechanicznej, sterowanie lotem drona quadrotor stanowi złożony problem z zakresu automatyki i robotyki, wynikający z jego naturalnej niestabilności dynamicznej (2).

Utrzymanie stabilnego lotu wymaga ciągłego monitorowania orientacji drona w przestrzeni oraz odpowiedniego sterowania prędkościami obrotowymi poszczególnych silników. W praktyce oznacza to konieczność zastosowania algorytmów sterowania działających w czasie rzeczywistym, zdolnych do kompensowania zakłóceń oraz błędów pomiarowych (3). Jednym z najczęściej stosowanych rozwiązań w tego typu systemach jest regulator PID, który dzięki swojej prostocie i skuteczności znajduje szerokie zastosowanie w systemach sterowania obiektów dynamicznych.

Niniejsza praca koncentruje się na praktycznym aspekcie implementacji systemu sterowania i stabilizacji lotu drona quadrotor, zrealizowanego z wykorzystaniem regulatora PID oraz mikrokontrolera klasy embedded.

1.1. Wprowadzenie

Dron quadrotor jest bezzałogowym statkiem powietrznym wyposażonym w cztery silniki napędzające śmigła, rozmieszczone symetrycznie względem środka konstrukcji. (1) Sterowanie ruchem drona odbywa się poprzez odpowiednie zwiększanie lub zmniejszanie prędkości obrotowej poszczególnych wirników, co pozwala na realizację ruchów w trzech osiach: przechylenia (roll), pochylenia (pitch) oraz obrotu wokół osi pionowej (yaw)(patrz Rys. 1). Dodatkowo możliwa jest kontrola wysokości lotu poprzez zmianę sumarycznej siły nośnej generowanej przez wszystkie silniki.



Rysunek 1: Wizualizacja sposobu obrotu drona w osiach Roll, Pitch i Yaw

Źródło 1: [https://www.researchgate.net/profile/Venkatapathy-](https://www.researchgate.net/profile/Venkatapathy-Prithiviraj/publication/280573614/figure/fig1/AS:391604515950593@1470377200482/Yaw-pitch-and-roll-rotations-of-a-Quadcopter.png)

[Prithiviraj/publication/280573614/figure/fig1/AS:391604515950593@1470377200482/Yaw-pitch-and-roll-rotations-of-a-Quadcopter.png](https://www.researchgate.net/profile/Venkatapathy-Prithiviraj/publication/280573614/figure/fig1/AS:391604515950593@1470377200482/Yaw-pitch-and-roll-rotations-of-a-Quadcopter.png)

Jedną z głównych trudności związanych ze sterowaniem dronem quadrotor jest jego niestabilność. (1) W przeciwieństwie do klasycznych statków powietrznych, quadrotor nie posiada naturalnych właściwości stabilizujących, takich jak skrzydła czy powierzchnie sterowe. Nawet niewielkie zakłócenia, takie jak podmuchy powietrza lub błędy pomiarowe czujników, mogą prowadzić do utraty stabilności lotu. Z tego względu niezbędne jest zastosowanie układu sterowania, który w sposób ciągły, bieżący koryguje orientację drona na podstawie danych z czujników inercyjnych. (4)

1.2. Cel pracy

Celem niniejszej pracy było zaprojektowanie oraz implementacja systemu sterowania i stabilizacji lotu drona typu quadrotor z wykorzystaniem regulatora PID. Opracowany system umożliwia stabilny lot drona sterowanego ręcznie, przy jednoczesnym kompensowaniu odchylen od zadanej orientacji oraz zakłóceń zewnętrznych. Dodatkowym celem pracy było praktyczne sprawdzenie działania regulatora PID w rzeczywistym układzie embedded, opartym na mikrokontrolerze STM32 (5).

1.3. Zakres pracy

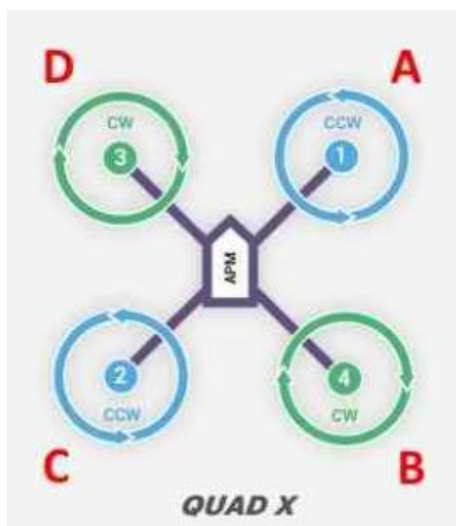
Zakres pracy obejmuje implementację systemu sterowania dronem quadrotor przeznaczonych do sterowania ręcznego z wykorzystaniem zewnętrznego nadajnika. W pracy zastosowano regulator PID odpowiedzialny za stabilizację kątów orientacji drona w trakcie lotu. Świadomie zrezygnowano z implementacji funkcji autonomicznego lotu, takich jak nawigacja czy planowanie trajektorii, koncentrując się na podstawowych mechanizmach stabilizacji i sterowania w czasie rzeczywistym. Takie podejście pozwoliło na skupienie się na kluczowych zagadnieniach związanych z regulacją i praktyczną implementacją algorytmu PID.

2. Stabilizacja lotu drona quadrotor

Stabilizacja lotu jest jednym z kluczowych zagadnień związanych z eksploatacją dronów wielowirnikowych (2). Dron typu quadrotor, ze względu na swoją konstrukcję, nie posiada naturalnych właściwości stabilizujących, co oznacza, że bez odpowiedniego układu sterowania nie jest w stanie utrzymać stabilnej orientacji w przestrzeni. Każde zakłócenie, takie jak zmiana obciążenia, nierównomierna praca silników czy wpływ otoczenia, prowadzi do odchylenia od położenia równowagi. Z tego względu stabilizacja lotu musi być realizowana w sposób ciągły, w oparciu o dane pomiarowe oraz algorytmy sterowania działające w czasie rzeczywistym (3).

2.1. Dron quadrotor

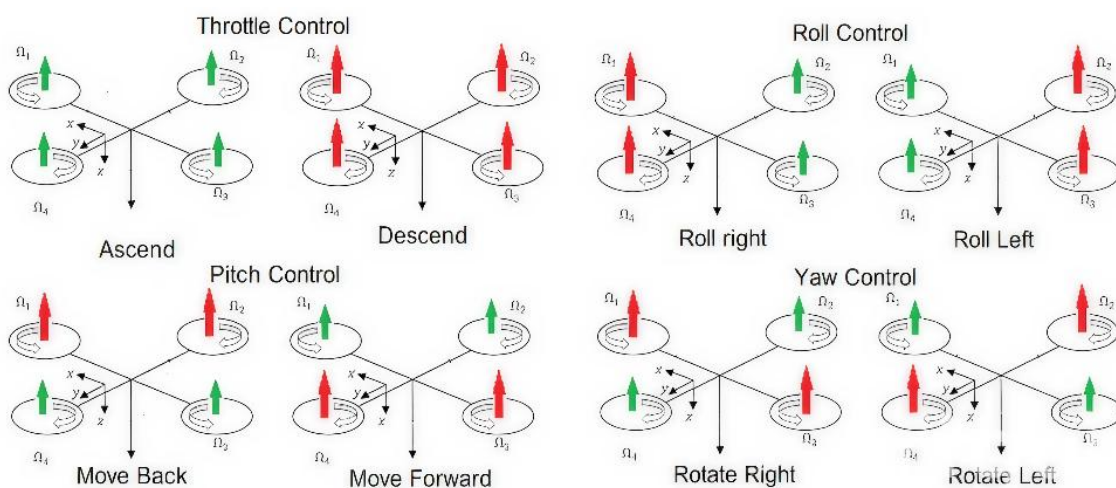
Quadrotor jest bezzałogowym statkiem powietrznym wyposażonym w cztery silniki napędzające śmigła, rozmieszczone symetrycznie względem środka konstrukcji. (1) Najczęściej spotykanym rozwiązaniem jest konfiguracja typu „X”, w której osie symetrii drona są przesunięte względem osi kadłuba. Każdy z silników generuje siłę nośną, której wartość zależy od prędkości obrotowej śmigła.



Rysunek 2: Wizualizacja rozmieszczenia silników i kierunków ich obrotu w konfiguracji "X"

Źródło 2: https://www.reddit.com/r/Multicopter/comments/hz5jkh/motors_on_quad_mislabeled_in_ardupilot/?tl=pl

Sterowanie ruchem drona odbywa się poprzez odpowiednie różnicowanie prędkości obrotowych poszczególnych silników. Zmiana sumarycznej siły nośnej pozwala na kontrolę wysokości lotu, natomiast różnice w ciągu pomiędzy parami silników umożliwiają realizację ruchów obrotowych wokół osi roll, pitch oraz yaw. Obrót wokół osi yaw realizowany jest dodatkowo poprzez zastosowanie przeciwnych kierunków obrotu śmigieł (zostało to zaznaczone na Rysunku 3.), co pozwala na kompensację momentów reakcyjnych.



Rysunek 3: Wizualizacja sposobu kontroli osi Roll, Pitch, Yaw

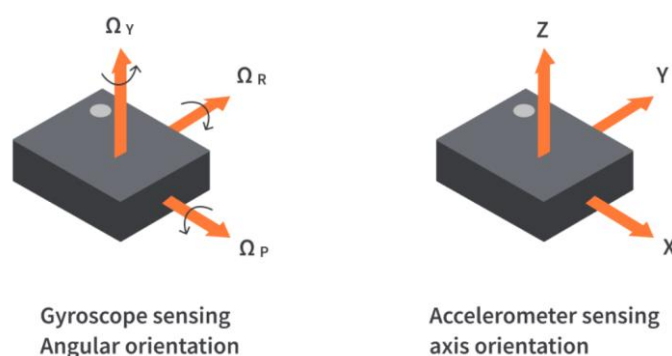
Źródło 3: https://www.researchgate.net/figure/Propeller-speeds-in-throttle-roll-pitch-and-yaw-control-Higher-speed-is-represented-by_fig1_352208829

Z punktu widzenia sterowania, dron quadrotor jest obiektem dynamicznym o silnych sprzężeniach pomiędzy osiami ruchu. Oznacza to, że zmiana sterowania w jednej osi może wpływać na zachowanie drona w pozostałych osiach, co dodatkowo utrudnia proces stabilizacji (4)

2.2. Czujniki inercyjne w stabilizacji lotu

Podstawowym źródłem informacji o orientacji drona w przestrzeni są czujniki inercyjne, najczęściej zintegrowane w postaci jednostki IMU (Inertial Measurement Unit). Tego typu układy zawierają zestaw czujników umożliwiających pomiar przyspieszeń liniowych oraz prędkości kątowych, które są niezbędne do określenia aktualnego położenia i orientacji drona (6).

Dane z czujników inercyjnych są wykorzystywane przez algorytm sterowania do obliczania odchyłeń od zadanej orientacji. Na ich podstawie generowane są sygnały sterujące, które korygują prędkości obrotowe silników. W praktyce oznacza to, że poprawność i szybkość działania systemu stabilizacji w dużej mierze zależy od jakości i częstotliwości odczytu danych z czujników.



Rysunek 4: Wizualizacja ułożenia osi w wybranych czujnikach inercyjnych

Źródło 4: <https://www.circuitbread.com/ee-faq/how-do-accelerometers-and-gyroscopes-work>

2.2.1. Akcelerometr

Akcelerometr jest czujnikiem służącym do pomiaru przyspieszenia liniowego wzdłuż trzech osi. W przypadku dronów quadrotor akcelerometr wykorzystywany jest przede wszystkim do określania orientacji względem wektora grawitacji (1) (6). Na podstawie pomiarów możliwe jest oszacowanie pochylenia i przechylenia drona.

Należy jednak zauważyć, że akcelerometr jest wrażliwy na zakłócenia wynikające z ruchu drona, takie jak przyspieszenia powstające podczas manewrów. Z tego względu jego

wskazania nie zawsze dokładnie odzwierciedlają rzeczywistą orientację drona, szczególnie w dynamicznych warunkach lotu.

2.2.2. Żyroskop

Żyroskop umożliwia pomiar prędkości kątowej wokół trzech osi obrotu. W systemach stabilizacji dronów jest on podstawowym czujnikiem wykorzystywanym do szybkiego wykrywania zmian orientacji (6). Dzięki wysokiej częstotliwości próbkowania, żyroskop pozwala na szybkie reagowanie układu sterowania na nagłe odchylenia.

Wadą żyroskopu jest dryft pomiarowy, polegający na narastaniu błędu orientacji w czasie. Z tego powodu dane z żyroskopu często wymagają korekcji, np. poprzez wykorzystanie informacji z akcelerometru. W niniejszej pracy czujniki inercyjne wykorzystywane są w sposób umożliwiający stabilizację orientacji drona w czasie rzeczywistym, bez stosowania zaawansowanych algorytmów estymacji.

2.3. Pojęcie stabilizacji lotu

Stabilizacja lotu drona polega na utrzymywaniu jego orientacji w pobliżu zadanych wartości, pomimo działania zakłóceń zewnętrznych oraz niedoskonałości elementów wykonawczych (3). W praktyce oznacza to ciągłe porównywanie aktualnych kątów orientacji z wartościami zadanymi oraz generowanie sygnałów sterujących minimalizujących powstały błąd.

W niniejszej pracy do realizacji stabilizacji lotu zastosowano regulator PID, który umożliwia efektywne tłumienie odchyłeń orientacji drona. Zastosowanie tego typu regulatora pozwala na uzyskanie stabilnego zachowania układu przy relatywnie niewielkiej złożoności implementacyjnej, co czyni go odpowiednim rozwiązaniem dla systemów embedded.

3. Regulator PID w systemie sterowania dronem

Jednym z najczęściej stosowanych algorytmów sterowania w systemach regulacji jest regulator PID (Proportional–Integral–Derivative). Ze względu na swoją prostą strukturę, niewielkie wymagania obliczeniowe oraz wysoką skuteczność w szerokiej klasie obiektów dynamicznych, regulator PID znajduje powszechne zastosowanie w automatyce przemysłowej, robotyce oraz systemach sterowania dronami (2). W przypadku drona

quadrotor regulator PID umożliwia stabilizację orientacji poprzez ciągłą korekcję prędkości obrotowych silników na podstawie błędu pomiędzy wartością zadaną a aktualnym pomiarem.

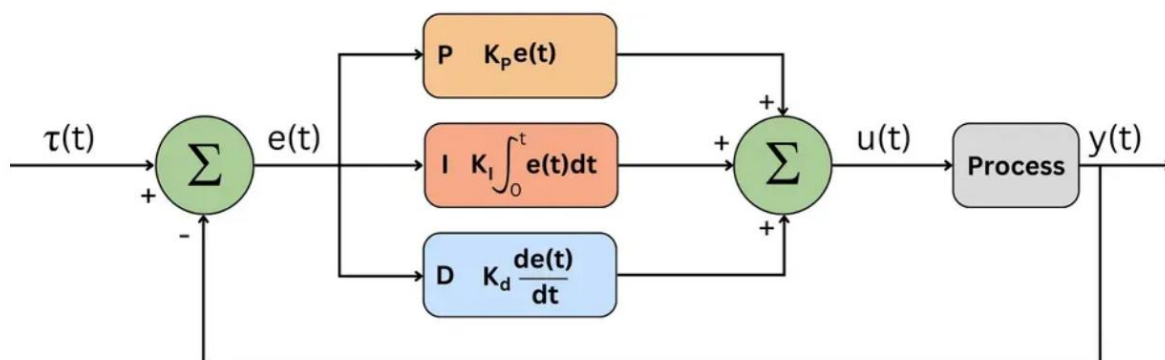
W niniejszej pracy regulator PID został wykorzystany jako podstawowy element systemu stabilizacji lotu, odpowiadający za tłumienie odchyłeń kątów orientacji drona w osiach roll, pitch oraz yaw.

3.1. Zasada działania regulatora PID

Regulator PID jest regulatorem działającym w pętli sprzężenia zwrotnego. Jego zadaniem jest minimalizacja błędu regulacji, definiowanego jako różnica pomiędzy wartością zadaną a wartością zmierzoną (2). Na podstawie tego błędu regulator generuje sygnał sterujący, który wpływa na obiekt regulacji, w tym przypadku na prędkości obrotowe silników drona.

Sygnał wyjściowy regulatora PID jest sumą trzech składowych:

- członu proporcjonalnego (P),
- członu całkującego (I),
- członu różniczkującego (D).



Rysunek 5: Schemat działania regulatora PID

Źródło 5: <https://www.wevolver.com/article/understanding-process-control-an-important-technique-for-industrial-processes>

Każda z tych składowych odpowiada za inny aspekt zachowania układu regulacji, a ich odpowiednie dobranie pozwala uzyskać stabilną i szybką odpowiedź systemu.

3.2. Człony regulatora PID

3.2.1. Człon proporcjonalny (P)

Człon proporcjonalny generuje sygnał sterujący proporcjonalny do aktualnej wartości błędu regulacji. Oznacza to, że im większe jest odchylenie od wartości zadanej, tym większa jest reakcja regulatora. Zastosowanie członu proporcjonalnego pozwala na szybkie reagowanie na zmiany orientacji drona.

Zbyt duże wzmocnienie członu proporcjonalnego może jednak prowadzić do niestabilności układu, natomiast zbyt małe powoduje wolną reakcję i niedostateczną korekcję błędu. Dlatego dobór parametru proporcjonalnego ma kluczowe znaczenie dla poprawnej pracy regulatora.

3.2.2. Człon całkujący (I)

Człon całkujący odpowiada za eliminację błędu ustalonego, który może występować w układach wykorzystujących wyłącznie regulację proporcjonalną. Działa on poprzez akumulowanie błędu regulacji w czasie i generowanie sygnału sterującego zależnego od sumy wcześniejszych błędów.

W systemach stabilizacji dronów zastosowanie członu całkującego pozwala na dokładniejsze utrzymanie zadanej orientacji. Należy jednak zauważyć, że nadmierne wzmocnienie tego członu może prowadzić do zjawiska narastania całki (4), co objawia się opóźnioną reakcją układu i możliwą utratą stabilności. Z tego względu parametr całkujący powinien być dobierany ostrożnie.

3.2.3. Człon różniczkujący (D)

Człon różniczkujący reaguje na szybkość zmian błędu regulacji, co pozwala przewidywać przyszłe zachowanie układu. Jego zadaniem jest tłumienie gwałtownych zmian oraz ograniczanie oscylacji, które mogą pojawić się w wyniku działania członu proporcjonalnego i całkującego.

W praktycznych implementacjach regulatora PID w systemach embedded człon różniczkujący bywa stosowany w ograniczonym zakresie, ponieważ jest wrażliwy na szumy

pomiarowe pochodzące z czujników. W niniejszej pracy człon różniczkujący został wykorzystany w celu poprawy stabilności dynamicznej układu.

3.3. Zastosowanie regulatora PID w stabilizacji drona

W systemie sterowania dronem quadrotor regulator PID wykorzystywany jest do stabilizacji kątów orientacji. Dla każdej z osi ruchu (roll, pitch oraz yaw) wyznaczany jest błąd regulacji, a następnie obliczana jest wartość sygnału sterującego. Sygnały te są wykorzystywane do modyfikacji prędkości obrotowych poszczególnych silników w taki sposób, aby zminimalizować odchylenie od wartości zadanych (4).

Regulatory PID pracują w pętli sterowania o stałej częstotliwości, co umożliwia bieżącą korekcję orientacji drona w czasie rzeczywistym. Zastosowanie oddzielnych regulatorów dla poszczególnych osi pozwala na uproszczenie struktury algorytmu oraz ułatwia proces strojenia parametrów.

3.4. Strojenie regulatora PID

Dobór parametrów regulatora PID ma istotny wpływ na zachowanie całego systemu sterowania (4). W niniejszej pracy parametry regulatora zostały dobrane metodą prób i błędów, polegającą na stopniowej modyfikacji wzmocnień poszczególnych członów oraz obserwacji reakcji drona.

Proces strojenia rozpoczęto od ustawienia wyłącznie członu proporcjonalnego, a następnie stopniowo wprowadzano człon całkujący oraz różniczkujący. Takie podejście pozwoliło na uzyskanie stabilnej pracy układu przy zachowaniu akceptowalnej dynamiki odpowiedzi. Metoda prób i błędów, mimo swojej prostoty, okazała się wystarczająca do osiągnięcia założonych celów pracy.

4. Projekt systemu sterowania drona

Celem projektu systemu sterowania było stworzenie spójnej architektury sprzętowo-programowej umożliwiającej stabilny lot drona quadrotor sterowanego ręcznie. Projekt uwzględniał zarówno dobór odpowiednich komponentów elektronicznych, jak i

zaprojektowanie algorytmu sterowania realizującego stabilizację orientacji drona w czasie rzeczywistym. Szczególny nacisk położono na niezawodność działania systemu korekcji położenia.

4.1. Założenia projektowe

Na etapie projektowania systemu sterowania przyjęto następujące założenia:

- dron ma być sterowany ręcznie za pomocą nadajnika RC,
- system sterowania ma realizować stabilizację kątów orientacji drona,
- algorytm sterowania oparty jest na regulatorze PID,
- system pracuje w czasie rzeczywistym na mikrokontrolerze klasy embedded,
- rozwiązanie ma mieć możliwie prostą i przejrzystą strukturę,
- elektronika sterująca zostanie zintegrowana na dedykowanej płycie PCB.

Świadomie zrezygnowano z implementacji funkcji autonomicznych, takich jak nawigacja czy planowanie trajektorii, aby skoncentrować się na podstawowych zagadnieniach stabilizacji i sterowania.

4.2. Architektura systemu sterowania

System sterowania drona składa się z kilku podstawowych bloków funkcjonalnych, które współpracują ze sobą w ramach zamkniętej pętli sterowania. Centralnym elementem systemu jest mikrokontroler STM32, który odpowiada za przetwarzanie danych z czujników, realizację algorytmu sterowania oraz generowanie sygnałów sterujących dla regulatorów ESC.

Do systemu sterowania należą następujące elementy:

- mikrokontroler STM32 jako jednostka obliczeniowa,
- czujnik inercyjny IMU dostarczający danych o orientacji drona,
- odbiornik RC przekazujący sygnały sterowania od użytkownika,
- regulatory ESC sterujące silnikami,
- silniki wraz ze śmigłami

4.3. Struktura algorytmu sterowania

Algorytm sterowania realizowany jest w postaci pętli sterowania wykonywanej cyklicznie z ustaloną częstotliwością. W każdej iteracji pętli wykonywane są następujące kroki:

- odczyt danych z czujników inercyjnych,
- obliczenie aktualnych kątów orientacji drona,
- wyznaczenie błędu regulacji względem wartości zadanych,
- obliczenie sygnałów sterujących z wykorzystaniem regulatorów PID,
- miksowanie sygnałów sterujących dla poszczególnych silników,
- generowanie sygnałów PWM sterujących regulatorami ESC.

Taka struktura algorytmu umożliwia ciągłą korekcję orientacji drona oraz szybką reakcję na zakłócenia zewnętrzne.

4.4. Projekt elektroniki sterującej

Istotnym elementem projektu systemu było zaprojektowanie dedykowanej elektroniki sterującej w postaci własnej płytki drukowanej PCB. Zastosowanie własnej płytki pozwoliło na integrację wszystkich kluczowych komponentów systemu sterowania w jednej strukturze oraz ograniczenie liczby połączeń przewodowych, co ma istotne znaczenie w przypadku konstrukcji latających.

Projekt płytki PCB uwzględnia rozmieszczenie pinów mikrokontrolera STM32, złącz do podłączenia czujnika inercyjnego, odbiornika RC oraz wyjść PWM do regulatorów ESC. Dodatkowo przewidziano linie zasilania oraz elementy pomocnicze niezbędne do poprawnej pracy systemu.

Na etapie projektowym skupiono się na funkcjonalnym podziale sygnałów oraz czytelnej strukturze połączeń. Szczegóły dotyczące implementacji płytki PCB, jej budowy oraz rozmieszczenia elementów zostały przedstawione w kolejnym rozdziale pracy.

4.5. Wykorzystane narzędzia programowe

W trakcie realizacji projektu wykorzystano następujące narzędzia programowe:

- Autodesk Fusion 360 – wykorzystany do projektowania elementów mechanicznych oraz modeli 3D konstrukcji drona, w tym rozmieszczenia komponentów elektronicznych i weryfikacji dopasowania elementów.
- AutoCAD – użyty do przygotowania rysunków technicznych oraz dokumentacji 2D elementów konstrukcyjnych.
- STM32CubeMX – zastosowany do konfiguracji mikrokontrolera STM32, przypisania wyprowadzeń, ustawienia interfejsów komunikacyjnych oraz wygenerowania kodu inicjalizacyjnego projektu.
- STM32CubeIDE – wykorzystany jako główne środowisko programistyczne do implementacji algorytmów sterowania, obsługi czujników, komunikacji oraz debugowania i programowania mikrokontrolera.
- Biblioteka STM32 HAL – użyta do obsługi peryferiów mikrokontrolera, takich jak timery, interfejsy UART, I2C oraz generacja sygnałów PWM.
- PuTTY – wykorzystany jako terminal szeregowy do monitorowania komunikacji UART, podglądu danych diagnostycznych oraz weryfikacji poprawności działania oprogramowania w trakcie testów.
- JLCPCB (7) – platforma wykorzystana do produkcji zaprojektowanej płytki drukowanej PCB na podstawie przygotowanych plików produkcyjnych (Gerber).

5. Implementacja systemu sterowania

W niniejszym rozdziale przedstawiono realizację zaprojektowanego systemu sterowania drona quadrotor. Opis obejmuje zarówno implementację sprzętową, w tym dedykowaną płytkę PCB oraz sposób połączenia elementów systemu, jak i implementację programową algorytmu sterowania oraz stabilizacji lotu.

5.1. Implementacja sprzętowa

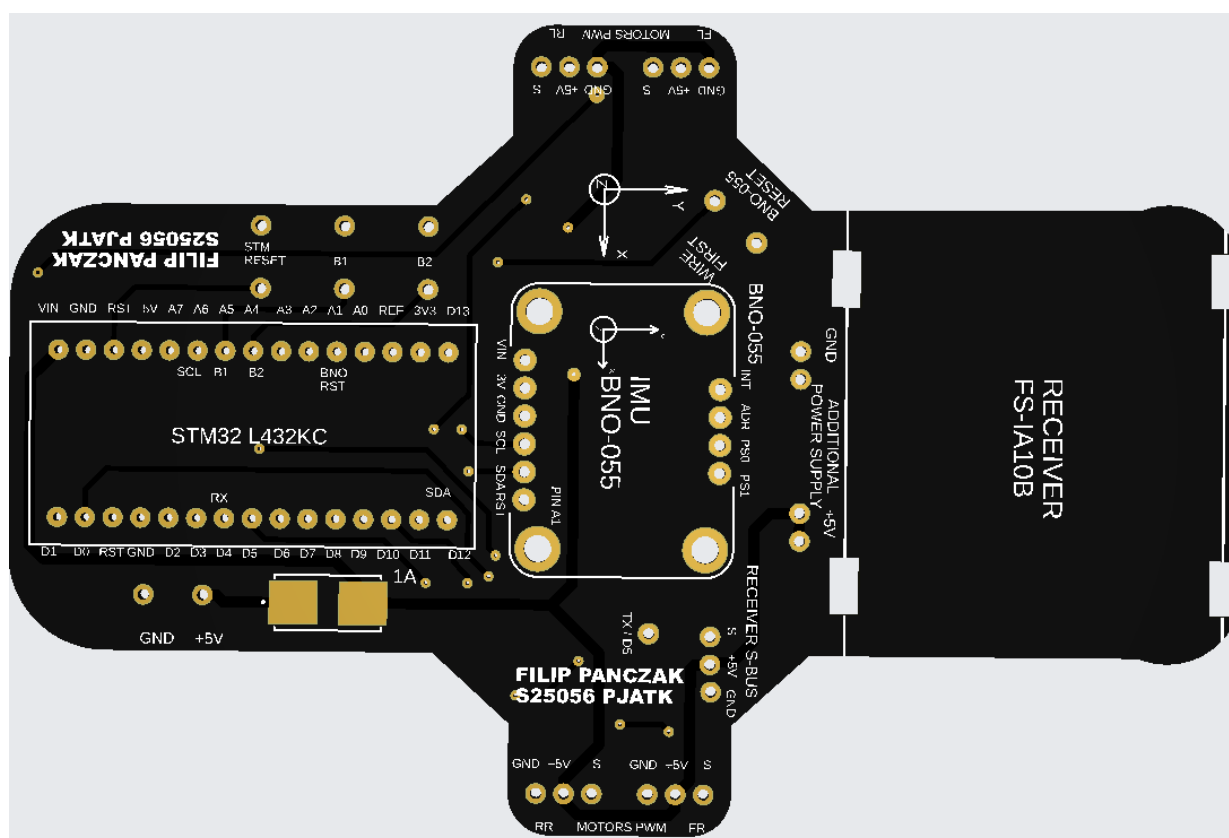
5.1.1. Dedykowana płytką PCB sterownika lotu

W celu integracji elementów systemu sterowania zaprojektowano dedykowaną płytkę drukowaną PCB, która pełni rolę sterownika lotu drona. Zastosowanie własnej płytki pozwoliło na uporządkowanie połączeń elektrycznych, zmniejszenie liczby przewodów oraz zwiększenie niezawodności całego systemu.

Projekt płytki PCB został wykonany w środowisku Autodesk Fusion. Podczas projektowania uwzględniono rozmieszczenie mikrokontrolera STM32, złącz do podłączenia czujnika inercyjnego BNO055, odbiornika RC FS-IA10B oraz wyjść PWM do regulatorów ESC. Dodatkowo zaplanowano zasilanie +5V oraz elementy pomocnicze niezbędne do poprawnej pracy układu.

Płytką PCB stanowi centralny punkt systemu sterowania, do którego podłączone są wszystkie pozostałe elementy elektroniczne drona.

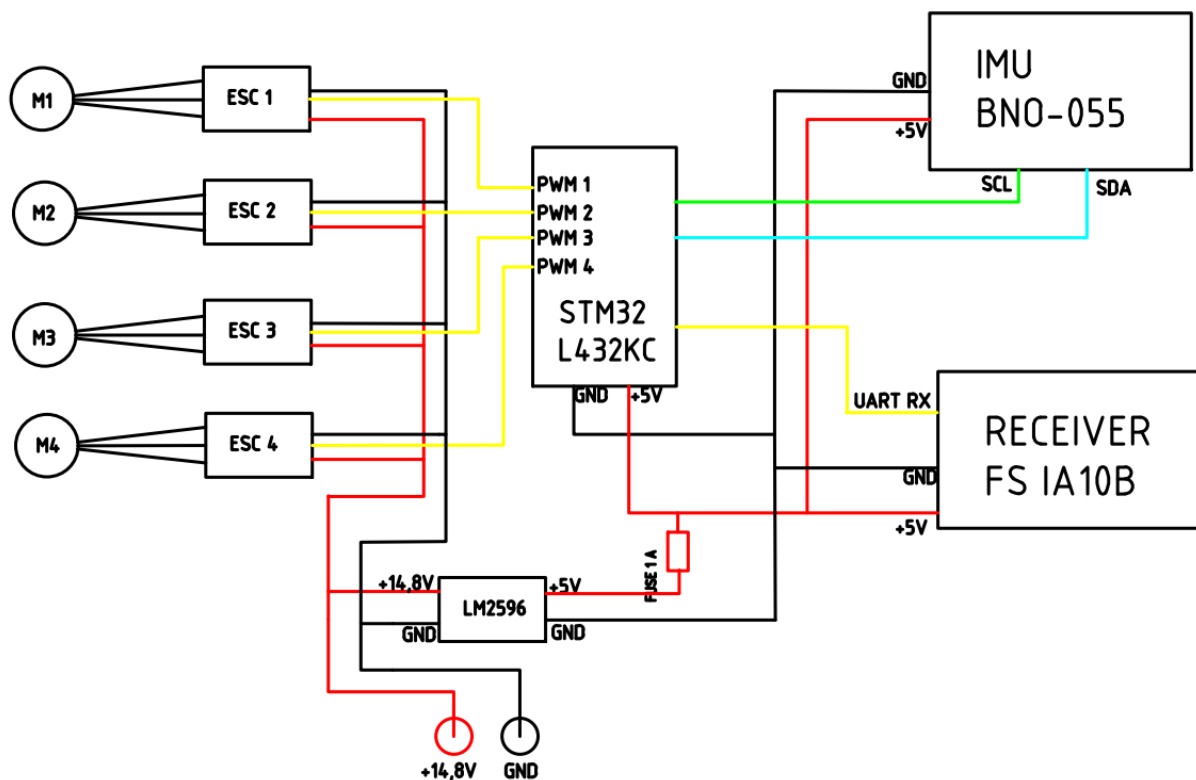
Dobre komponenty czy złącza zostały przylutowane do płytki co zwiększa jakość połączeń.



Rysunek 6: Dedykowana, autorska płytką PCB do sterowania dronem

Źródło 6: Opracowanie własne

Każdy z elementów układu elektronicznego został przedstawiony na poniższym schemacie.



Rysunek 7: Diagram połączeń elektronicznych

Źródło 7: Opracowanie własne

TUTAJ DODAC PRZYPISY DO TYCH KOMPONENTÓW Z LINKAMI

Lista zastosowanych komponentów w projekcie:

- Mikrokontroler STM32 Nucleo L432KC
- Czujnik IMU BNO055
- Odbiornik RC FlySky FS-IA10B
- Nadajnik RC FlySky FS-I6X
- Regulatory ESC BLHeli_S 20A (4 szt.)
- Silniki bezszczotkowe (BLDC) BR2205 2300KV (4 szt.)
- Śmigła 5calowe (4 szt.)
- Przetwornica step-down LM2596 (3,2–35 V, 3 A)
- Zasilacz laboratoryjny NDN DF173005C (używane 14,8 V – testy stanowiskowe)
- Dedykowana płytki PCB sterownika lotu
- Rama drona quadrotor FPV F330

5.1.2 Zasilanie układu

Całość elektroniki sterującej zasilana jest stabilizowanym napięciem +5 V, uzyskiwanym z przetwornicy typu step-down LM2596. Przetwornica obniża napięcie z głównej linii zasilania do poziomu odpowiedniego dla komponentów na dedykowanej płycie PCB.

Podczas testów prowadzonych na stanowisku testowym, zarówno przetwornica LM2596, jak i regulatory ESC zasilane są z laboratoryjnego zasilacza NDN DF173005C, ustawionego na napięcie 14,8 V. Takie rozwiązanie umożliwia bezpieczne testowanie systemu bez użycia akumulatora pokładowego oraz łatwą kontrolę parametrów zasilania. Wszystkie elementy systemu posiadają wspólną masę, co zapewnia poprawne odniesienie sygnałów logicznych i stabilną pracę układu.

5.1.3. Mikrokontroler STM32L432KC

Mikrokontroler STM32L432KC pełni rolę centralnej jednostki sterującej systemem, jest głównym elementem całego układu. Odpowiada on za realizację algorytmu sterowania, dekodowanie ramek UART (iBus) z odbiornika RC, przetwarzanie danych z czujników oraz generowanie sygnałów sterujących PWM dla regulatorów ESC.

Wybór mikrokontrolera STM32L432KC został podyktowany kilkoma istotnymi czynnikami technicznymi oraz praktycznymi. Układ ten oparty jest na rdzeniu ARM Cortex-M4, który oferuje wystarczającą moc obliczeniową do realizacji algorytmów sterowania w czasie rzeczywistym, w tym regulatorów PID oraz obsługi komunikacji z czujnikami inercyjnymi.

Mikrokontroler ten charakteryzuje się również bogatym zestawem peryferiów. Pozwala to na jednoczesną obsługę odbiornika RC (iBus), czujnika IMU oraz regulatorów ESC bez konieczności stosowania dodatkowych układów zewnętrznych.

Istotnym argumentem był także niski pobór mocy, typowy dla serii STM32L, co ma znaczenie w systemach zasilanych bateryjnie.

5.1.4. Czujnik inercyjny IMU BNO055

Czujnik inercyjny BNO055 dostarcza informacji o orientacji drona w przestrzeni, które są niezbędne do realizacji algorytmu stabilizacji lotu. Czujnik połączony jest z mikrokontrolerem STM32 za pomocą magistrali I2C, umożliwiającej dwukierunkową transmisję danych pomiędzy układami.

Zasilanie czujnika realizowane jest napięciem +5 V z tej samej linii zasilania co mikrokontroler. Masa czujnika połączona jest ze wspólną masą systemu. W prezentowanym systemie nie wykorzystano linii przerwania INT czujnika inercyjnego BNO055. Dane z czujnika odczytywane są cyklicznie w głównej pętli sterowania metodą pollingu, co przy odpowiednio dobranej częstotliwości pętli zapewnia aktualne informacje o orientacji drona.

5.1.5. Odbiornik RC FS-IA10B

Odbiornik RC FS-IA10B służy do odbioru sygnałów sterujących wysyłanych z nadajnika przez operatora drona (8). Odbiornik przekazuje dane do mikrokontrolera w postaci cyfrowej, z wykorzystaniem protokołu iBUS.

Połączenie z mikrokontrolerem STM32 realizowane jest poprzez interfejs UART, przy czym wykorzystywana jest wyłącznie linia odbiorcza UART RX. Odbiornik zasilany jest napięciem +5 V z przetwornicy step-down, a jego masa połączona jest ze wspólną masą systemu.

5.1.6. Regulatory ESC BLHeli_S 20A

Sterowanie napędem drona realizowane jest za pomocą czterech regulatorów ESC, z których każdy odpowiada za sterowanie jednym silnikiem bezszczotkowym. Regulatory ESC odbierają sygnały sterujące PWM generowane przez mikrokontroler STM32. Każdy sygnał PWM odpowiada za regulację prędkości obrotowej jednego silnika.

Regulatory ESC zasilane są bezpośrednio z głównej linii zasilania o napięciu 14,8 V, pochodzącej z zasilacza laboratoryjnego. Linie masy regulatorów ESC połączone są ze wspólną masą systemu sterowania, co zapewnia poprawne odniesienie sygnałów PWM.

5.1.7. Silniki bezszczotkowe (BLDC) BR2205 2300KV

Silniki bezszczotkowe stanowią element wykonawczy systemu sterowania i odpowiadają za generowanie siły nośnej oraz momentów obrotowych drona. Każdy silnik podłączony jest do odpowiedniego regulatora ESC, który steruje jego prędkością obrotową na podstawie sygnału PWM otrzymanego z mikrokontrolera.

Mikrokontroler STM32 nie steruje silnikami bezpośrednio, a jedynie pośrednio poprzez regulatory ESC, co jest standardowym rozwiązaniem w konstrukcjach dronów wielowirnikowych.

5.2. Implementacja programowa

Implementacja programowa systemu sterowania drona quadrotor została zrealizowana na mikrokontrolerze STM32 z wykorzystaniem środowiska STM32CubeIDE oraz biblioteki sprzętowej HAL (Hardware Abstraction Layer). Oprogramowanie odpowiada za realizację algorytmu stabilizacji lotu w czasie rzeczywistym, przetwarzanie danych z czujników inercyjnych, obsługę sygnałów sterowania z nadajnika RC oraz generowanie sygnałów sterujących dla regulatorów ESC.

Kod programu został zaprojektowany w sposób modułowy, co ułatwia jego analizę, modyfikację oraz dalszy rozwój. W obecnej wersji oprogramowania system realizuje stabilizację kątów orientacji drona w osiach roll oraz pitch z wykorzystaniem regulatora PID.

5.2.1. Biblioteka HAL (Hardware Abstraction Layer)

W projekcie sterownika drona wykorzystano bibliotekę HAL (Hardware Abstraction Layer), dostarczaną przez producenta mikrokontrolera. Biblioteka ta stanowi warstwę pośrednią pomiędzy sprzętem a kodem aplikacyjnym, upraszczając dostęp do zasobów mikrokontrolera oraz ujednolicając sposób ich obsługi.

Głównym celem biblioteki HAL jest abstrakcja sprzętowa, czyli ukrycie szczegółów związanych z bezpośrednim sterowaniem rejestrami mikrokontrolera. Dzięki temu programista może korzystać z gotowych funkcji wysokiego poziomu, co znacząco przyspiesza proces tworzenia oprogramowania oraz zmniejsza ryzyko popełnienia błędów.

5.2.1.1. Struktura i sposób działania biblioteki HAL

Biblioteka HAL składa się z zestawu modułów odpowiadających poszczególnym peryferiom mikrokontrolera, takim jak:

- GPIO (wejścia/wyjścia cyfrowe)
- TIM (timery)
- I2C, SPI, UART (interfejsy komunikacyjne)
- system przerwań i zegarów

Każdy z modułów zawiera:

- funkcje inicjalizujące (np. konfiguracja trybu pracy, częstotliwości, preskalerów),
- funkcje obsługi (np. wysyłanie i odbiór danych)
- funkcje przerwań i callbacki

Takie podejście zapewnia czytelność kodu oraz wyraźne oddzielenie konfiguracji sprzętu od logiki sterowania.

5.2.1.2. Zastosowanie HAL w projekcie dron

W niniejszym projekcie biblioteka HAL została wykorzystana m.in. do:

- obsługi interfejsu I2C, używanego do komunikacji z czujnikiem inercyjnym IMU
- konfiguracji timerów, generujących przerwania dla pętli głównej
- sterowania sygnałami PWM, odpowiedzialnymi za regulację prędkości obrotowej silników
- obsługi portów GPIO, np. do sygnalizacji stanu pracy układu za pomocą diod LED

Dzięki zastosowaniu HAL możliwe było skupienie się na implementacji algorytmu sterowania (PID) oraz logice działania systemu, bez konieczności ręcznej konfiguracji rejestrów sprzętowych.

5.2.1.3. Zalety wykorzystania biblioteki HAL

Do głównych zalet zastosowania biblioteki HAL w projekcie należą:

- zwiększona czytelność i przejrzystość kodu,
- krótszy czas tworzenia oprogramowania,
- łatwiejsze debugowanie i utrzymanie projektu,
- większa przenośność kodu pomiędzy różnymi modelami mikrokontrolerów,
- zgodność z narzędziami konfiguracyjnymi (np. generowanie kodu inicjalizacyjnego).

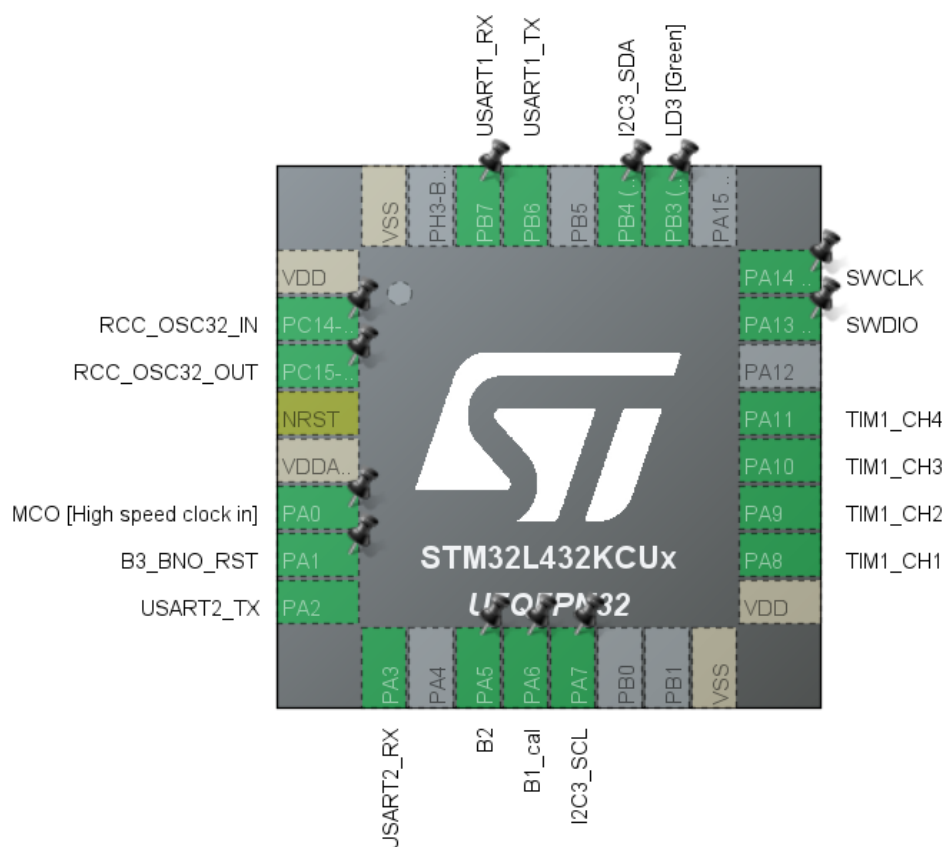
5.2.2. Środowisko programistyczne i konfiguracja systemu

Oprogramowanie sterujące zostało opracowane w środowisku STM32CubeIDE, które umożliwia konfigurację peryferiów mikrokontrolera oraz generację kodu inicjalizacyjnego.

Na etapie konfiguracji systemu skonfigurowano następujące peryferia mikrokontrolera:

- interfejs I2C do komunikacji z czujnikiem inercyjnym BNO055
- interfejs UART do odbioru danych z odbiornika RC w protokole iBUS
- drugi interfejs UART do odczytywania wartości serial portu poprzez port USB
- timery sprzętowe do generacji sygnałów PWM dla regulatorów ESC
- timer systemowy odpowiedzialny za synchronizację pętli sterowania
- Porty GPIO do diod oraz przycisków debugujących

Konfiguracja zegarów mikrokontrolera została dobrana w taki sposób, aby zapewnić stabilną pracę systemu oraz odpowiednią częstotliwość wykonywania algorytmu sterowania.



Rysunek 8: Konfiguracja Pinów mikrokontrolera STM32L432KC w programie STM32CubeMX

Źródło 8: Opracowanie własne

5.2.3. Struktura programu

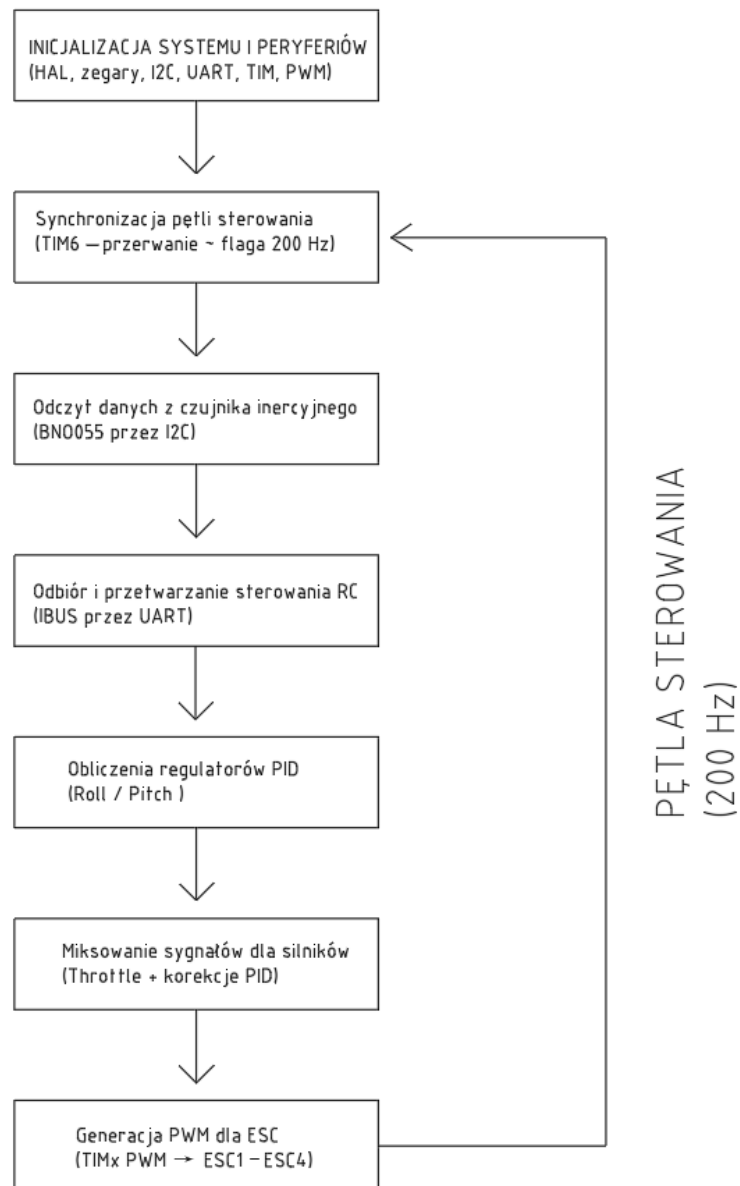
Struktura programu oparta jest na klasycznym modelu aplikacji embedded, składającym się z funkcji inicjalizacyjnych, pętli głównej (while) oraz procedur obsługi przerwań. Główna logika sterowania realizowana jest w pętli głównej programu, natomiast synchronizacja czasowa odbywa się przy użyciu sprzętowego timera.

Program podzielono logicznie na następujące obszary:

- inicjalizacja systemu i peryferiów,
- odczyt danych z czujnika inercyjnego,
- odbiór i przetwarzanie sygnałów sterowania RC,
- obliczenia regulatorów PID,
- miksowanie sygnałów sterujących dla silników,
- generacja sygnałów PWM dla regulatorów ESC.

Takie podejście umożliwia czytelne rozdzielenie funkcji systemu oraz ułatwia dalsze modyfikacje kodu.

DIAGRAM BLOKOWY STRUKTURY PROGRAMU



Rysunek 9: Schemat blokowy pętli sterowania

Źródło 9: Opracowanie własne

5.2.4. Synchronizacja pętli sterowania

Algorytm sterowania dronem wykonywany jest z ustaloną częstotliwością, niezależną od czasu wykonywania pętli głównej programu. W tym celu wykorzystano sprzętowy timer TIM6, który generuje przerwy okresowe z częstotliwością 200 Hz.

W procedurze obsługi przerwania timera ustawiana jest flaga sterująca, informująca pętlę główną o konieczności wykonania kolejnej iteracji algorytmu sterowania.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        control_loop_flag = 1;    // flaga dla pętli głównej
    }
}
```

Pętla główna sprawdza stan tej flagi i realizuje obliczenia regulatora PID wyłącznie w momencie jej ustawienia.

```
    // czeka na flagę z TIM6
    if (!control_loop_flag)
        continue;
    control_loop_flag = 0;
```

Takie rozwiązanie zapewnia:

- stałą częstotliwość pracy algorytmu sterowania,
- możliwość kontroli częstotliwości wykonywania pętli
- odporność na zmienne czasy wykonywania kodu.

5.2.5. Odczyt danych z czujnika inercyjnego

Czujnik inercyjny BNO055 komunikuje się z mikrokontrolerem STM32 za pośrednictwem magistrali I2C. W prezentowanym systemie czujnik pracuje w trybie dostarczającym bezpośrednio kąty orientacji w postaci kątów Eulera.

Dane z czujnika odczytywane są cyklicznie w każdej iteracji pętli sterowania. Odczytane wartości kątów wykorzystywane są jako sygnały pomiarowe dla regulatorów PID.

Zrezygnowano z implementacji zaawansowanych algorytmów fuzji danych, co pozwoliło na uproszczenie struktury programu oraz skupienie się na praktycznej implementacji systemu stabilizacji.

5.2.6. Odbiór i przetwarzanie sygnałów sterowania RC

5.2.6.1. Odbiornik RC i protokół iBUS

Sygnały sterowania od operatora drona odbierane są za pomocą odbiornika RC, który komunikuje się z mikrokontrolerem przy użyciu protokołu iBUS. Protokół ten wykorzystuje transmisję UART i umożliwia przesyłanie wielu kanałów sterujących w jednej ramce danych. W porównaniu do klasycznych sygnałów PWM, iBUS charakteryzuje się większą odpornością na zakłócenia oraz mniejszą ilością okablowania.

W prezentowanym systemie wykorzystana jest wyłącznie linia odbiorcza interfejsu UART, ponieważ komunikacja z odbiornikiem RC jest jednokierunkowa. Dane są przesyłane w postaci ramek o stałej długości, zawierających wartości kanałów sterujących zakodowane jako 16-bitowe liczby całkowite.

5.2.6.2. Odbiór danych UART z wykorzystaniem DMA

Do odbioru danych z odbiornika RC zastosowano mechanizm DMA (Direct Memory Access), który umożliwia bezpośrednie zapisywanie odebranych bajtów z interfejsu UART do bufora pamięci operacyjnej. Dzięki temu procesor nie jest angażowany w obsługę każdego pojedynczego bajtu danych, co znacząco redukuje jego obciążenie.

5.2.6.3. Dekodowanie ramek iBUS

Dekodowanie danych z odbiornika RC realizowane jest programowo poprzez przeszukiwanie bufora DMA w poszukiwaniu początku ramki iBUS. Ramka identyfikowana jest na podstawie charakterystycznego nagłówka bajtów 0x20 oraz 0x40. Po wykryciu potencjalnego początku ramki wykonywana jest weryfikacja sumy kontrolnej, która pozwala odrzucić błędne lub niekompletne dane.

Po poprawnej weryfikacji ramki następuje odczyt wartości poszczególnych kanałów sterujących, które zapisywane są do tablicy kanałów. Dodatkowo prowadzone są liczniki ramek poprawnych i błędnych, co umożliwia ocenę jakości transmisji podczas testów systemu (wykorzystane tutaj do debugowania).

Fragment funkcji odpowiedzialnej za dekodowanie ramek iBUS przedstawiono poniżej:

```

static void ibus_try_parse_from_dma(void)
{
    for (int start = 0; start <= (IBUS_DMA_BUF_SZ - IBUS_FRAME_SZ); start++)
    {
        const uint8_t *f = &ibus_dma_buf[start];
        if (f[0] != 0x20 || f[1] != 0x40)
            continue;
        uint16_t rx_chk = (uint16_t)f[30] | ((uint16_t)f[31] << 8);
        uint16_t calc = ibus_checksum(f);
        if (rx_chk != calc)
        {
            ibus_frames_bad++;
            continue;
        }
        for (int ch = 0; ch < IBUS_CHANNELS; ch++)
        {
            int idx = 2 + ch * 2;
            ibus_ch[ch] = (uint16_t)f[idx] | ((uint16_t)f[idx + 1] << 8);
        }
        ibus_frames_ok++;
        return;
    }
}

```

5.2.6.4. Mapowanie kanałów sterujących

Odebrane wartości kanałów sterujących odpowiadają pozycjom drążków pilota RC i są mapowane na konkretne funkcje sterujące dronem. Wartości kanałów iBUS, typowo mieszczące się w zakresie od około 1000 do 2000, są następnie normalizowane i przekształcane do postaci sygnałów zadanych wykorzystywanych w algorytmie sterowania.



Rysunek 10: Osi przechyłu drona i ich przypisanie do pilota RC

Źródło 10: <https://coadroneflightschool.wordpress.com/lesson-5-10517/>

5.2.6.5. Zadawanie wartości sterujących z pilota

Na podstawie znormalizowanych sygnałów z aparatury RC wyznaczane są wartości zadane dla poszczególnych osi sterowania.

W przypadku osi roll oraz pitch sygnały z pilota odpowiadają zadanim kątom nachylenia, które stanowią wartości zadane dla regulatorów PID stabilizujących orientację drona.

Sterowanie osi yaw realizowane jest bez wykorzystania regulatora PID – sygnał z pilota wpływa bezpośrednio na proces miksowania prędkości obrotowych silników, umożliwiając ręczny obrót drona wokół osi pionowej.

Sygnał throttle pełni rolę bazowej wartości ciągu, na którą nakładane są korekty wynikające z działania regulatorów PID oraz sterowania yaw. Takie podejście zapewnia intuicyjne sterowanie dronem oraz stabilną pracę układu sterowania przy jednoczesnym zachowaniu prostoty algorytmu.

5.2.7. Implementacja regulatora PID

Stabilizacja orientacji drona realizowana jest z wykorzystaniem regulatorów PID, odpowiedzialnych za bieżącą korektę kątów przechylenia (roll) oraz pochylenia (pitch). Dla każdej z tych osi zaimplementowano osobną instancję regulatora, pracującą w oparciu o

aktualne dane pomiarowe z czujnika inercyjnego oraz wartości zadane określone przez użytkownika.

Regulator PID stanowi kluczowy element algorytmu sterowania i pełni rolę głównej logiki decyzyjnej systemu. Na podstawie obliczonego błędu regulacji wyznaczone są sygnały korekcyjne, które następnie wpływają na prędkości obrotowe poszczególnych silników drona. W przeciwieństwie do rozwiązań symulacyjnych, regulator PID w prezentowanym systemie pracuje w rzeczywistym układzie embedded i wykonywany jest cyklicznie z ustaloną częstotliwością 200 Hz.

Algorytm regulatora uwzględnia trzy składowe: proporcjonalną (P), całkującą (I) oraz różniczkującą (D). W implementacji zastosowano mechanizm ograniczania narastania całki (anti-windup) poprzez ograniczenie wartości integratora. Człon różniczkujący liczony jest na podstawie zmian sygnału pomiarowego (tzw. derivative on measurement), co zmniejsza wrażliwość regulatora na skoki wartości zadanej oraz szumy pomiarowe.

```
float PID_Update(PID_t* pid, float setpoint, float measurement, float dt)
{
    if (dt <= 0.0f) return 0.0f;

    // błąd regulacji
    float e = setpoint - measurement;

    // człon proporcjonalny
    float p = pid->kp * e;

    // człon całkujący (anti-windup przez ograniczenie integratora)
    pid->integrator += pid->ki * e * dt;
    pid->integrator = clampf(pid->integrator, pid->i_min, pid->i_max);

    // człon różniczkujący (liczony na podstawie pomiaru)
    float d = 0.0f;
    if (pid->first_run)
    {
        pid->prev_meas = measurement;
        pid->first_run = false;
    }
    else
    {
        float meas_dot = (measurement - pid->prev_meas) / dt;
        d = -pid->kd * meas_dot;
        pid->prev_meas = measurement;
    }
}
```

```

float out = p + pid->integrator + d;
return clampf(out, pid->out_min, pid->out_max);
}

```

5.2.8. Wyznaczanie pomiarów orientacji

W każdej iteracji pętli sterowania odczytywane są aktualne kąty orientacji drona na podstawie danych dostarczanych przez czujnik inercyjny BNO055. Czujnik zwraca orientację w postaci kątów Eulera, które następnie mapowane są na osie sterowania roll i pitch.

Ze względu na fizyczną orientację czujnika na konstrukcji drona, w algorytmie sterowania zastosowano korektę znaków osi. W szczególności oś pitch została odwrócona (znakiem minus przed wartością), aby zachować spójność konwencji znaków pomiędzy pomiarem a reakcją układu sterowania. Takie podejście zapewnia intuicyjne zachowanie drona podczas stabilizacji oraz upraszcza proces strojenia regulatorów PID.

```

float roll_meas = (float)euler.y;    // ROLL
float pitch_meas = -(float)euler.z;   // PITCH (odwrócona oś)
// float yaw_meas = (float)euler.x;   // YAW (nieużywany)

```

5.2.9. Warstwa bezpieczeństwa i reset regulatorów

W pętli sterowania zaimplementowano prostą warstwę bezpieczeństwa, której zadaniem jest zapobieganie niepożądanemu zachowaniu regulatorów PID przy niskim ciągu silników. W przypadku, gdy sygnał gazu (throttle) spadnie poniżej ustalonego progu,

regulatory PID osi roll i pitch są resetowane, a do regulatorów ESC przekazywane są minimalne wartości sterujące.

```
if (throttle_us < 1050)
{
    PID_Reset(&pid_roll);
    PID_Reset(&pid_pitch);
    mixer_update(0.0f, 0.0f, 0.0f, 1000);
    continue;
}
```

Reset regulatorów w takim stanie zapobiega narastaniu składowej całkującej w sytuacji, gdy silniki nie generują wystarczającego ciągu. Rozwiązanie to zwiększa bezpieczeństwo uruchamiania systemu oraz eliminuje ryzyko gwałtownej reakcji drona po ponownym zwiększeniu gazu.

5.2.10. Procedura kalibracji regulatorów ESC

W celu zapewnienia poprawnej współpracy mikrokontrolera z regulatorami ESC zaimplementowano procedurę kalibracji zakresu sygnałów PWM. Kalibracja umożliwia dopasowanie minimalnej i maksymalnej szerokości impulsu generowanego przez mikrokontroler do zakresu rozpoznawanego przez regulatory ESC.

Procedura kalibracji uruchamiana jest wyłącznie podczas startu systemu. Wciśnięcie dedykowanego przycisku w momencie podania zasilania powoduje przejście programu w tryb kalibracji ESC. W normalnym trybie pracy procedura ta nie jest wykonywana, co eliminuje ryzyko przypadkowej kalibracji podczas działania systemu.

```
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_SET)
{
    esc_calibrate_all();

    // po zakończeniu kalibracji program zostaje zatrzymany
    while (1)
    {
        HAL_GPIO_TogglePin(LD3_GPIO_Port, LD3_Pin);
        HAL_Delay(200);
    }
}
```

```
}  
}
```

Funkcja odpowiedzialna za kalibrację regulatorów ESC:

```
void esc_calibrate_all(void)  
{  
    motors_set_all_us(2000); // maksymalna wartość PWM  
    HAL_Delay(3000);         // czas na wykrycie MAX przez ESC  
  
    motors_set_all_us(1000); // minimalna wartość PWM  
    HAL_Delay(3000);         // czas na zapis MIN  
  
    motors_set_all_us(1000); // bezpieczny stan wyjść  
}
```

Zatrzymanie programu po kalibracji wymusza odłączenie zasilania przed ponownym uruchomieniem systemu i zwiększa bezpieczeństwo obsługi układu.

5.2.11. Obliczenia regulatorów PID i miksowanie sygnałów

Po wyznaczeniu pomiarów orientacji oraz wartości zadanych wykonywane są obliczenia regulatorów PID dla osi roll i pitch. Każdy regulator generuje sygnał korekcyjny, który następnie przekazywany jest do modułu miksowania.

```
float u_roll  = PID_Update(&pid_roll,  roll_sp,  roll_meas,  CTRL_DT);  
float u_pitch = PID_Update(&pid_pitch, pitch_sp, pitch_meas, CTRL_DT);  
float u_yaw   = yaw_rate_sp;    // YAW bez pida poki co  
  
mixer_update(u_roll, u_pitch, u_yaw, throttle_us);
```

Moduł miksowania łączy sygnały korekcyjne z wartością throttle, roll, pitch oraz yaw pilota RC i wyznacza końcowe wartości sterujące dla czterech silników drona. Obliczone sygnały przekazywane są do regulatorów ESC w postaci sygnałów PWM generowanych przez sprzętowe timery mikrokontrolera STM32. Takie rozwiązanie zapewnia płynną regulację prędkości silników oraz szybką reakcję układu na odchylenia orientacji.

5.2.12. Warunek minimalnego położenia gazu

W systemie sterowania zaimplementowano zabezpieczenie uniemożliwiające uruchomienie algorytmu sterowania oraz silników w przypadku, gdy drążek gazu nadajnika RC nie znajduje się w położeniu minimalnym. System nie rozpoczyna pracy, dopóki sygnał gazu z odbiornika RC nie spadnie poniżej zdefiniowanego progu bezpieczeństwa.

Takie rozwiązanie pełni funkcję blokady startu i zapobiega przypadkowemu uruchomieniu silników w sytuacji, gdy nadajnik znajduje się w niebezpiecznym stanie, na przykład po ponownym włączeniu zasilania lub utracie synchronizacji. Dopiero po ustawieniu drążka gazu w położeniu minimalnym możliwe jest przejście systemu w tryb normalnej pracy.

Zastosowanie blokady minimalnego położenia gazu zwiększa bezpieczeństwo obsługi drona oraz zapewnia przewidywalne zachowanie systemu podczas uruchamiania.

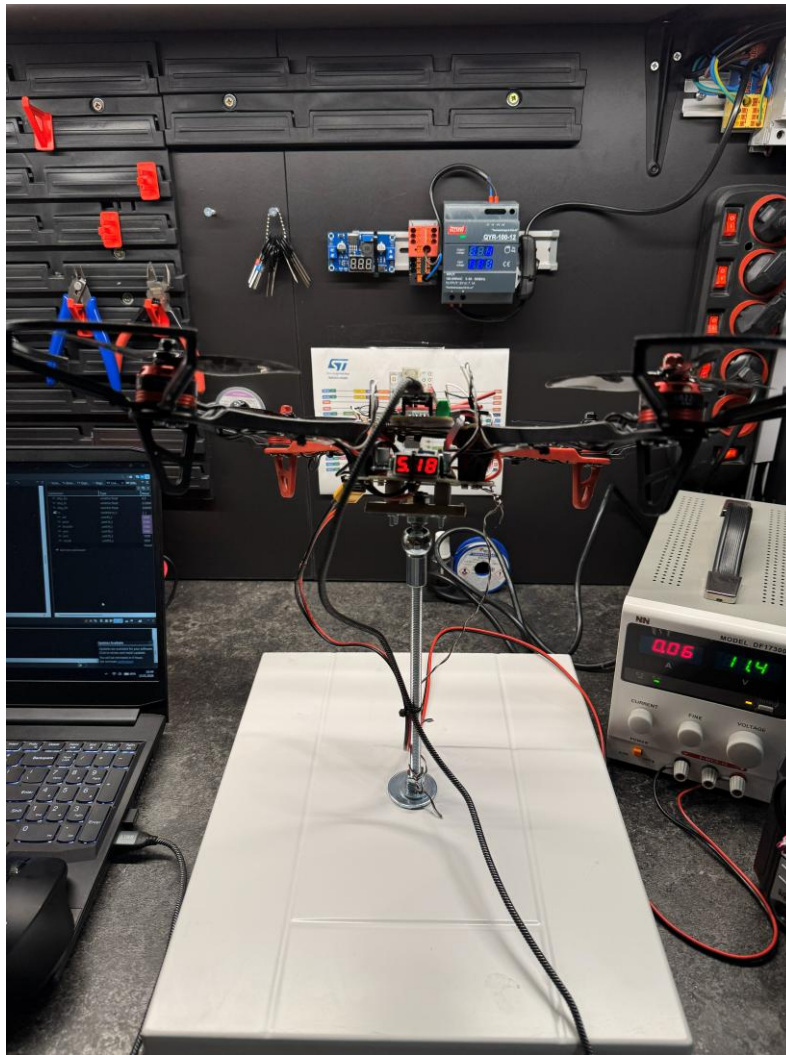
6. Testy działania systemu

Celem przeprowadzonych testów było zweryfikowanie poprawności działania zaimplementowanego systemu sterowania oraz ocena skuteczności regulatora PID w stabilizacji orientacji drona. Testy miały na celu sprawdzenie zachowania układu w warunkach zbliżonych do rzeczywistych, przy jednoczesnym zachowaniu wysokiego poziomu bezpieczeństwa oraz możliwości bezpośredniej obserwacji reakcji systemu.

Ze względu na etap realizacji projektu oraz charakter pracy inżynierskiej, testy przeprowadzono w warunkach stanowiskowych, bez wykonywania swobodnych lotów.

6.1. Stanowisko testowe

Testy systemu sterowania przeprowadzono na dedykowanym stanowisku testowym, w którym dron był zamocowany na przegubie kulowym w sposób umożliwiający ograniczony ruch obrotowy w osiach roll oraz pitch. Konstrukcja stanowiska pozwalała na swobodne wychylenia drona w zadanym zakresie kątowym.



Zastosowanie takiego stanowiska testowego umożliwiło:

- bezpieczne testowanie algorytmu stabilizacji bez ryzyka uszkodzenia konstrukcji,
- wymuszanie zakłóceń w postaci wychyleń drona,
- obserwację reakcji regulatora PID w czasie rzeczywistym,
- prowadzenie testów przy różnych nastawach regulatora.
- strojenie regulatora „na żywo”

Stanowisko umożliwia jednocześnie podłączenie drona poprzez interfejs USB (UART) dzięki czemu można bezproblemowo debugować kod w trakcie działania systemu a także korzystać z możliwości konsoli Serial Monitor.

6.1.2. Druga wersja stanowiska

W drugiej wersji stanowiska testowego zastosowano przegub kulowy o mocowaniu M8, zastępując wcześniejszy przegub M6. Zmiana ta spowodowała zwiększenie momentu bezwładności układu wynikające z większej masy oraz większego promienia rozłożenia masy względem osi obrotu. W efekcie do uzyskania tego samego przyspieszenia kąтового w osiach roll i pitch wymagany był większy moment generowany przez silniki. Przy niezmiennych nastawach regulatora PID reakcja układu stała się wolniejsza i mniej dynamiczna. Dodatkowo zmiana przegubu wpłynęła na położenie środka ciężkości, zwiększając moment grawitacyjny podczas przechyłów. Konieczne było ponowne dostrojenie parametrów regulatora PID, co potwierdziło istotny wpływ zmian mechanicznych na działanie algorytmu sterowania.

6.2. Metodyka przeprowadzania testów

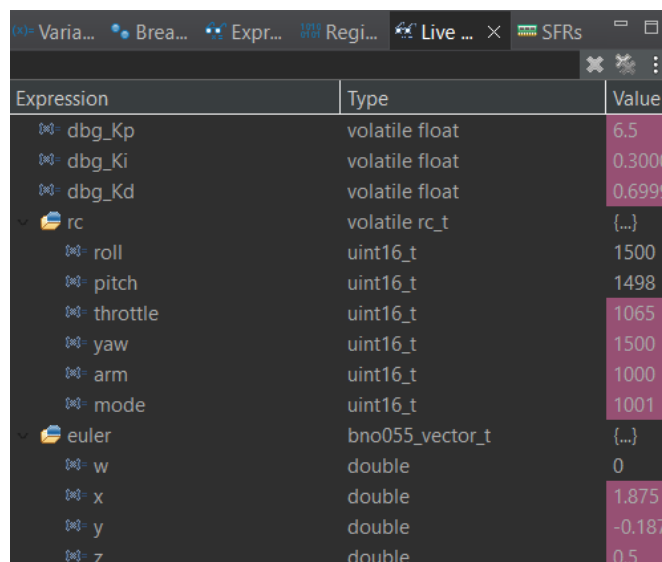
Testy realizowano w trybie sterowania ręcznego, przy czym wykorzystano jedynie wartość z kanału throttle. Wartości zadane dla osi roll oraz pitch ustawione były na zero, co odpowiadało dążeniu do utrzymania poziomej orientacji drona.

Testy polegały na:

- ręcznym wychylaniu drona w osi roll lub pitch,
- obserwacji odpowiedzi układu na zadane zakłócenie,
- analizie szybkości i płynności powrotu do położenia równowagi,
- ocenie wpływu zmian nastaw regulatora PID na zachowanie systemu.

6.3. Strojenie regulatora PID z wykorzystaniem zmiennych modyfikowanych w debuggerze

Dzięki stanowisku testowemu proces strojenia regulatora PID przeprowadzono z wykorzystaniem mechanizmu modyfikacji parametrów regulatora w czasie rzeczywistym, bez konieczności ponownej kompilacji i wgrywania programu do mikrokontrolera. W tym celu w kodzie programu zdefiniowano zmienne odpowiadające współczynnikom wzmocnień regulatora PID (K_p , K_i , K_d), które były obserwowane i modyfikowane bezpośrednio z poziomu debugera środowiska STM32CubeIDE.



Expression	Type	Value
dbg_Kp	volatile float	6.5
dbg_Ki	volatile float	0.3000
dbg_Kd	volatile float	0.6999
rc	volatile rc_t	{...}
roll	uint16_t	1500
pitch	uint16_t	1498
throttle	uint16_t	1065
yaw	uint16_t	1500
arm	uint16_t	1000
mode	uint16_t	1001
euler	bno055_vector_t	{...}
w	double	0
x	double	1.875
y	double	-0.187
z	double	0.5

Rysunek 11: Odczyt i modyfikacja zmiennych live w debugerze STM Cube IDE

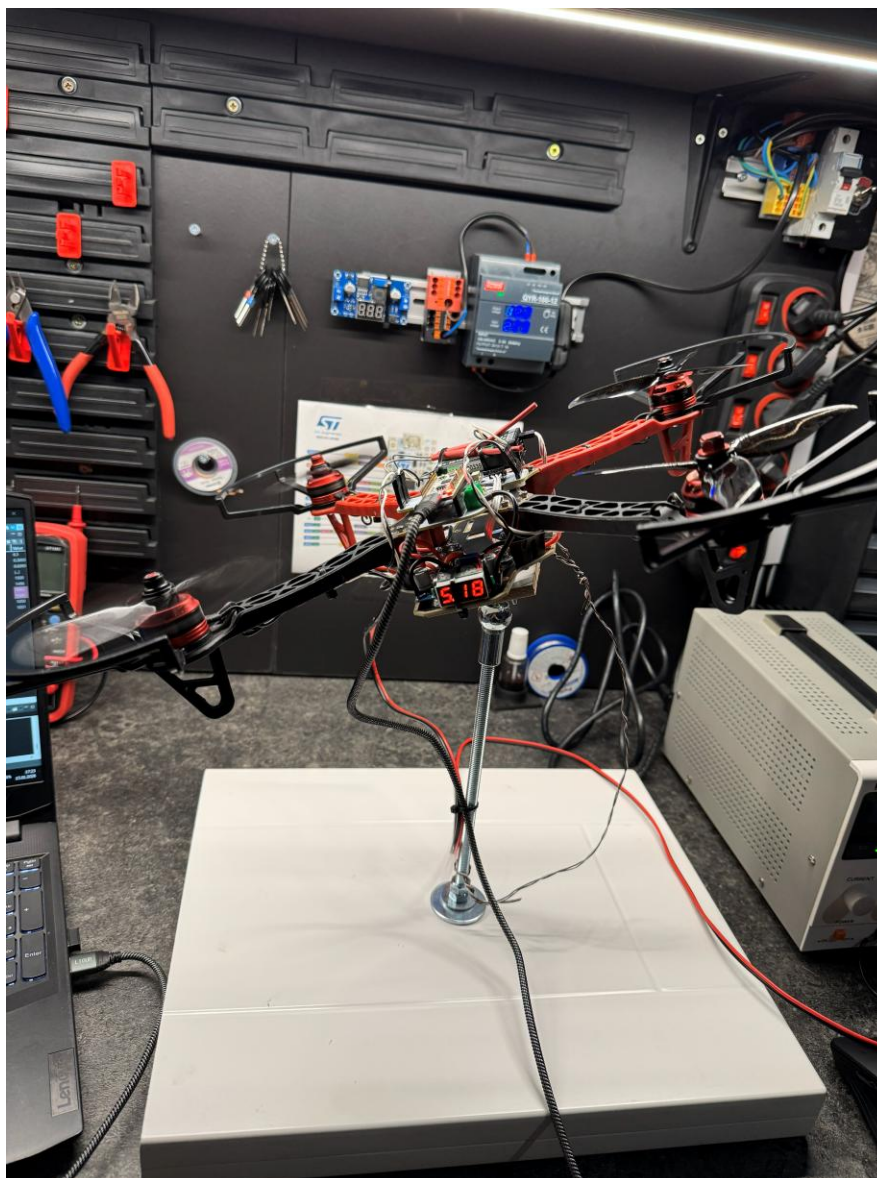
Źródło 11: Opracowanie własne

Podczas pracy systemu algorytm sterowania cyklicznie odczytywał aktualne wartości zmiennych odpowiadających nastawom regulatora i wykorzystywał je w kolejnych iteracjach pętli sterowania. Dzięki temu możliwe było natychmiastowe zaobserwowanie wpływu zmian parametrów regulatora na zachowanie drona, bez przerywania działania systemu

Dodatkowo, w przypadku zmiany nastaw regulatora podczas pracy systemu stosowano reset zmiennych stanu regulatora PID, w szczególności integratora. Pozwalało to na uniknięcie niepożądanego wpływu wcześniej zgromadzonej całki na zachowanie układu po zmianie parametrów, co miało istotne znaczenie dla poprawnej interpretacji wyników strojenia.

6.4. Obserwacje zachowania systemu

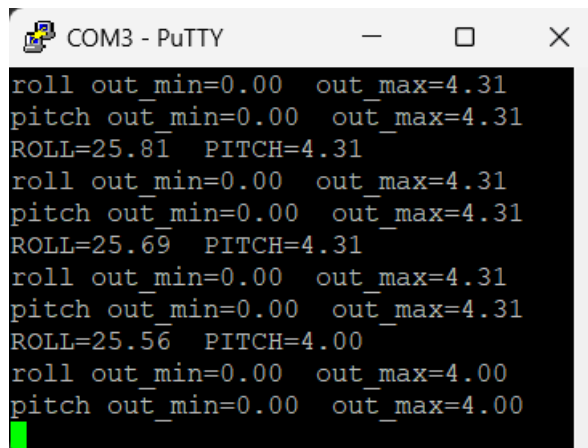
W trakcie testów stanowiskowych zaobserwowano wyraźny wpływ nastaw regulatora PID na zachowanie drona w osiach roll oraz pitch. Początkowe próby przeprowadzono z wstępnymi, nieoptymalnymi wartościami parametrów regulatora, co pozwoliło na ocenę charakterystycznych zjawisk występujących przy niewłaściwym doborze nastaw.



Rysunek 12: Testy Drona - start z pozycji maksymalnie wychylonej

Źródło 12: Opracowanie własne

Na powyższym zdjęciu widzimy sytuację gdzie dron startuje z pozycji wychylonej maksymalnie. Wartość throttle jest ustawiona na 1100 i dzięki tak małej prędkości silników możemy zauważyć pracę tylko dwóch silników. Przy takim wychyleniu obserwacje osi wyglądały tak jak na poniższym obrazie.



```
roll out_min=0.00 out_max=4.31
pitch out_min=0.00 out_max=4.31
ROLL=25.81 PITCH=4.31
roll out_min=0.00 out_max=4.31
pitch out_min=0.00 out_max=4.31
ROLL=25.69 PITCH=4.31
roll out_min=0.00 out_max=4.31
pitch out_min=0.00 out_max=4.31
ROLL=25.56 PITCH=4.00
roll out_min=0.00 out_max=4.00
pitch out_min=0.00 out_max=4.00
```

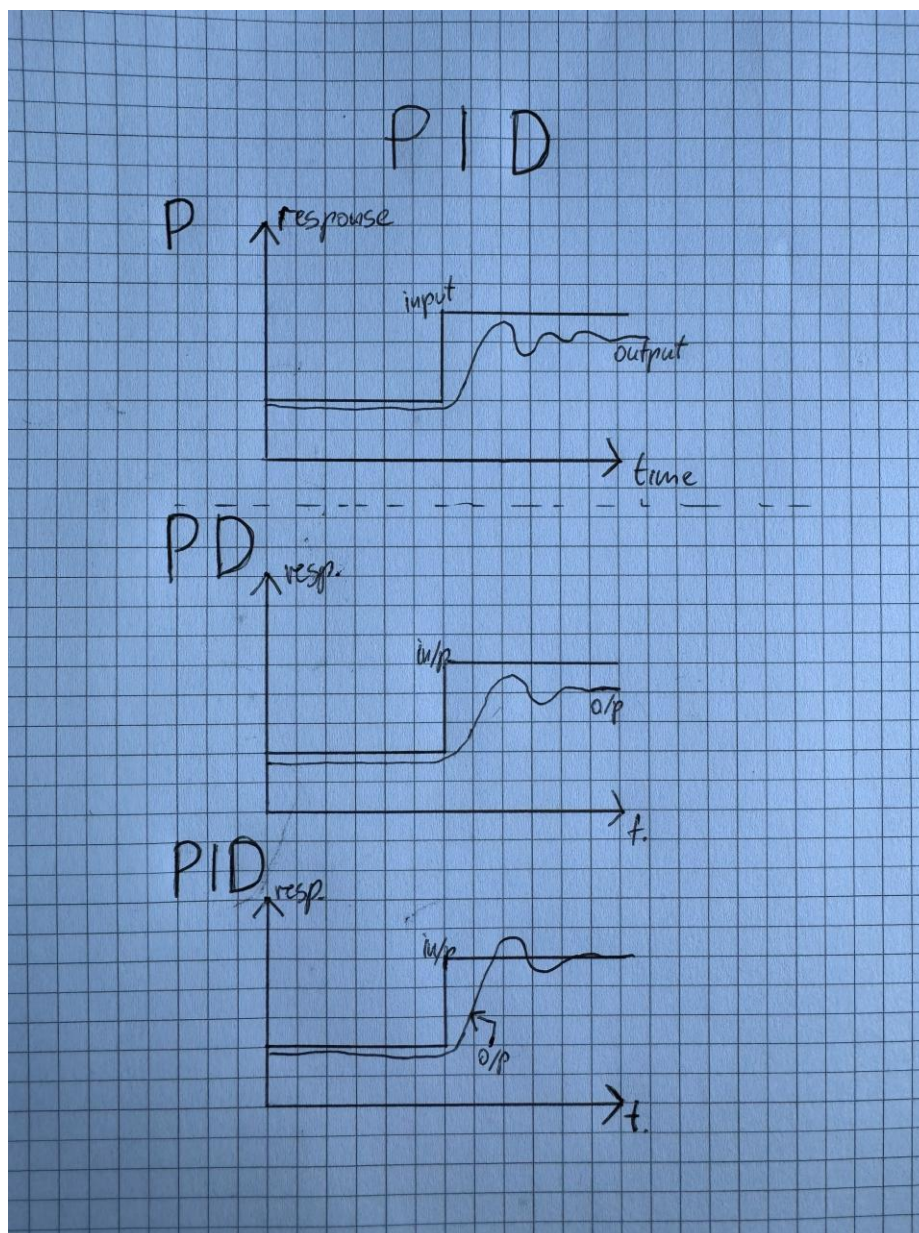
Rysunek 13: Obserwacje wychyleń osi ROLL i PITCH w aplikacji PuTTY podczas wychylenia maksymalnego drona na stanowisku testowym

Źródło 13: Opracowanie własne

Dodatkowo oprócz wychyleń osi Roll i Pitch w Serial Monitor obserwujemy aktualne minimalne i maksymalne wyjście korekt regulatora PID.

6.4.1. Wpływ poszczególnych członów regulatora na zachowanie drona

W regulatorze PID każdy człon(K_p , K_i oraz K_d) mają kluczowy wpływ na funkcjonowanie algorytmu. Na poniższej ilustracji zostało to zobrazowane.



Rysunek 14: Wpływ poszczególnych członów regulatora PID na jego wyjście

Źródło 14: Opracowanie własne

6.4.1.1. Zachowanie układu przy zbyt dużym wzmocnieniu K_p

Przy zbyt dużych wartościach współczynnika proporcjonalnego K_p układ reagował bardzo gwałtownie na wymuszone wychylenia. Obserwowano szybkie korekcje prowadzące do przeregulowania oraz powstawania oscylacji wokół położenia równowagi. Dron nie był w stanie ustabilizować się w jednym położeniu, a amplituda wychyleń utrzymywała się na stałym poziomie.

Zmniejszenie wartości K_p prowadziło do uspokojenia reakcji układu i stopniowego zaniku oscylacji.

6.4.1.2. Wpływ członu całkującego K_i

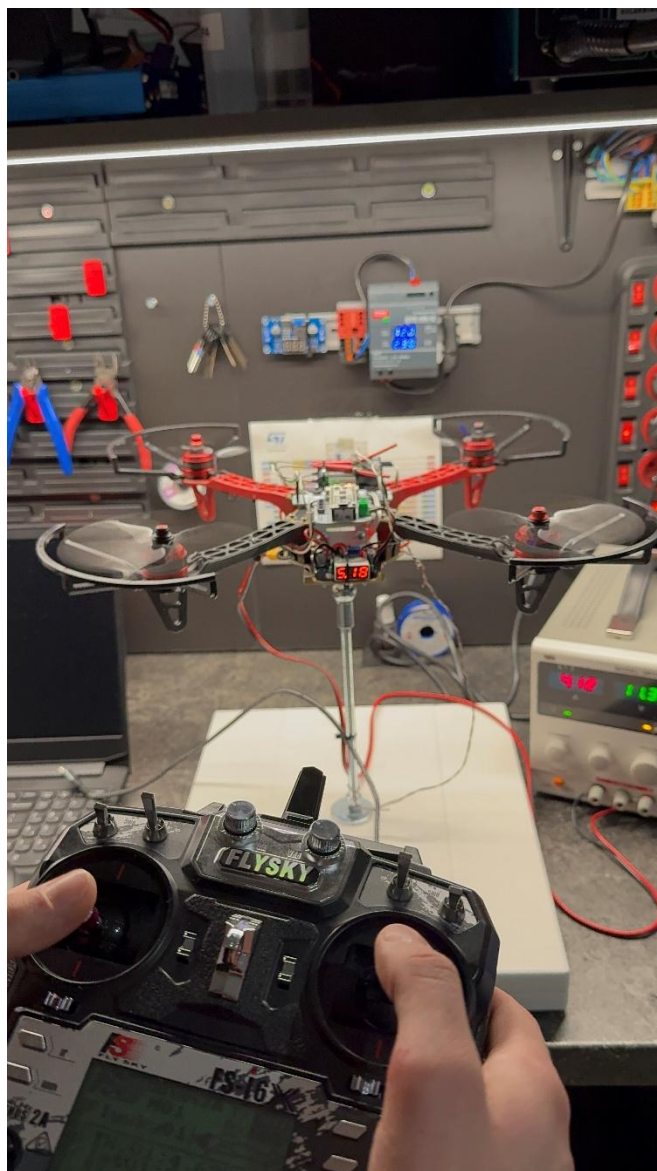
Wprowadzenie członu całkującego pozwoliło na redukcję niewielkiego błędu ustalonego, jednak przy zbyt dużej wartości współczynnika K_i obserwowano powolne narastanie wychylenia drona, nawet po ustaniu zakłócenia. Zjawisko to było szczególnie widoczne przy niskim sygnale gazu, gdy silniki nie generowały wystarczającego momentu korygującego.

Zastosowany mechanizm ograniczenia całki (anti-windup) oraz reset regulatora przy niskim gazie skutecznie ograniczały to zjawisko.

6.4.1.3 Wpływ członu różniczkującego K_d

Człon różniczkujący miał istotny wpływ na tłumienie gwałtownych zmian orientacji. Przy zbyt małej wartości K_d reakcja układu była agresywna, natomiast zbyt duże wzmocnienie prowadziło do nadmiernego tłumienia oraz zwiększonej wrażliwości na szumy pomiarowe. Objawiało się to drobnymi, szybkimi korektami, które nie poprawiały stabilności, a jedynie zwiększały nieregularność reakcji.

Dobór umiarkowanej wartości K_d pozwolił na uzyskanie płynnej odpowiedzi bez wyraźnych oscylacji.



Rysunek 15: Dron podczas pozycji (0,0) w trakcie działania z poprawnie dobranymi parametrami PID

Źródło 15: Opracowanie własne

Odczyt kątów orientacji w tym momencie wyglądał jak na poniższym obrazie.

```
COM3 - PuTTY
roll out_min=0.00 out_max=0.25
pitch out_min=0.00 out_max=0.25
ROLL=-0.31 PITCH=0.25
roll out_min=0.00 out_max=0.25
pitch out_min=0.00 out_max=0.25
ROLL=-0.31 PITCH=0.25
roll out_min=0.00 out_max=0.25
pitch out_min=0.00 out_max=0.25
ROLL=-0.31 PITCH=0.25
roll out_min=0.00 out_max=0.25
pitch out_min=0.00 out_max=0.25
```

6.4.2. Działanie mechanizmów bezpieczeństwa

Podczas testów potwierdzono poprawne działanie mechanizmów bezpieczeństwa systemu. Obniżenie sygnału gazu poniżej ustalonego progu powodowało reset regulatorów PID oraz przejście silników w stan minimalnego wysterowania. Po ponownym zwiększeniu gazu system rozpoczynał pracę w sposób przewidywalny, bez gwałtownych reakcji, niezależnie od wcześniejszych nastaw regulatora.

6.3. Ograniczenia przeprowadzonych testów

Przeprowadzone testy miały charakter stanowiskowy i nie obejmowały swobodnych lotów drona. Ograniczenie to wynikało z etapu realizacji projektu oraz koncentracji na weryfikacji podstawowej stabilizacji orientacji. W wyniku tego ograniczono wykorzystanie osi Yaw. Pomimo tych ograniczeń testy potwierdziły poprawność działania systemu sterowania oraz zasadność zastosowania regulatora PID w fizycznym układzie drona quadrotor.

7. Podsumowanie i wnioski

7.1. Cel i zakres wykonanej pracy

Celem niniejszej pracy było zaprojektowanie oraz wykonanie systemu stabilizacji drona z wykorzystaniem regulatora PID, działającego na rzeczywistym układzie fizycznego drona. W ramach pracy zrealizowano zarówno część sprzętową, jak i programową systemu sterowania, umożliwiającą stabilizację orientacji drona w osiach roll oraz pitch.

Zakres pracy obejmował dobór podzespołów, implementację oprogramowania na mikrokontrolerze STM32, integrację czujnika inercyjnego oraz regulatorów ESC, a także uruchomienie i testy systemu na stanowisku testowym.

7.2. Ocena działania systemu

Przeprowadzone testy potwierdziły, że zaprojektowany system sterowania działa poprawnie i spełnia założone funkcje. Regulator PID reagował na wymuszone wychylenia

drona, generując sygnały sterujące prowadzące do powrotu konstrukcji do położenia równowagi. Zastosowanie pętli sterowania o stałej częstotliwości zapewniło stabilną i powtarzalną pracę systemu. Wprowadzone zabezpieczenia, takie jak blokada uruchomienia przy niezerowym sygnale gazu oraz reset regulatorów PID przy niskim ciągu, zwiększyły bezpieczeństwo użytkownika i ułatwiły prowadzenie testów.

W trakcie badań zaobserwowano jednak, że przy położeniach bardzo bliskich wartościom zadany ($\text{roll}=0$, $\text{pitch}=0$) system nie zawsze osiągał pełne wyrównanie. W niektórych przypadkach konstrukcja zatrzymywała się w niewielkim odchyleniu od położenia równowagi. Zjawisko to wynikało ze zmiany właściwości mechanicznych stanowiska, w szczególności zwiększonego momentu bezwładności oraz przesunięcia środka ciężkości spowodowanych zastosowaniem większego przegubu kulowego. Przy małych odchyleniach generowane sygnały korekcyjne okazywały się czasami niewystarczające do całkowitego skompensowania działających momentów.

7.3. Wnioski z testów i strojenia regulatora PID

Proces strojenia regulatora PID miał istotny wpływ na zachowanie całego układu sterowania. Zaobserwowano, że zbyt duże wartości wzmocnienia proporcjonalnego prowadziły do występowania oscylacji, natomiast nadmierne wzmocnienie członu całkującego powodowało wolniejszą reakcję układu oraz pogorszenie jego stabilności.

Odpowiedni dobór członu różniczkującego pozwolił ograniczyć gwałtowne zmiany sygnałów sterujących i uzyskać płynniejszą odpowiedź systemu na zakłócenia. W trakcie testów potwierdzono również, że parametry regulatora muszą być dostosowane do właściwości mechanicznych stanowiska, w szczególności do momentu bezwładności i położenia środka ciężkości.

Strojenie regulatora w czasie rzeczywistym, z wykorzystaniem zmiennych modyfikowanych bezpośrednio w debuggerze, okazało się skutecznym i wygodnym rozwiązaniem w warunkach stanowiskowych, umożliwiając szybkie porównanie wpływu zmian nastaw na zachowanie układu.

7.4. Ograniczenia rozwiązania

Zaprojektowany system posiada pewne ograniczenia wynikające z przyjętych założeń projektu. Stabilizacja została zaimplementowana jedynie dla osi roll i pitch, natomiast oś yaw nie była regulowana. Można było natomiast kontrolować ją z poziomu pilota RC. Testy przeprowadzono wyłącznie na stanowisku testowym, bez realizacji lotów swobodnych.

Dodatkowo parametry regulatora PID były dobierane ręcznie, bez zastosowania automatycznych metod strojenia lub identyfikacji obiektu.

7.5. Możliwości dalszego rozwoju

W przyszłości opracowany system sterowania może zostać rozbudowany o stabilizację osi yaw, co pozwoli na pełną kontrolę orientacji drona w trzech osiach. Umożliwiłoby to realizację bardziej zaawansowanych manewrów oraz poprawę zachowania platformy w warunkach zakłóceń zewnętrznych. Dodatkowym kierunkiem rozwoju jest implementacja bardziej zaawansowanych algorytmów przetwarzania danych pomiarowych, w tym zwiększonej filtracji sygnałów pochodzących z czujnika inercyjnego, co mogłoby przyczynić się do dalszej poprawy stabilności i dokładności regulacji.

Kolejnym etapem rozwoju projektu mogłyby być testy w warunkach rzeczywistego lotu, które pozwoliłyby na weryfikację poprawności działania systemu poza stanowiskiem laboratoryjnym. Testy te umożliwiłyby również dostrojenie parametrów regulatorów oraz ocenę odporności układu na zakłócenia występujące podczas lotu.

Istotnym kierunkiem dalszych prac jest także integracja układu zasilania bezpośrednio na projektowanej płytce PCB. Zamiast stosowania zewnętrznego modułu przetwornicy LM2596 możliwe byłoby zaprojektowanie dedykowanego stabilizatora napięcia 5 V, dostosowanego do wymagań prądowych całego systemu. Takie rozwiązanie pozwoliłoby na zmniejszenie liczby połączeń zewnętrznych, poprawę estetyki układu oraz zwiększenie jego niezawodności.

Dodatkowo płytka mogłaby zostać wyposażona w elementy zabezpieczające, takie jak warystory, dławiki oraz filtry wejściowe, chroniące elektronikę przed przepięciami, zakłóceniami oraz skutkami zwarć. Integracja tych elementów bezpośrednio na płytce PCB zwiększyłaby odporność systemu na uszkodzenia oraz poprawiła bezpieczeństwo jego użytkowania.

Podsumowując, dalszy rozwój projektu może obejmować zarówno rozszerzenie funkcjonalności programowej, jak i udoskonalenie warstwy sprzętowej, prowadząc do powstania bardziej zwartego, niezawodnego i uniwersalnego systemu sterowania dronem.

ZAPYTAC PROMOTORA O BIBLIOTEKE BNO KTÓRĄ WZIAŁEM OD ZNAJOMEGO

BIBLIOGRAFIA

1. **Bouabdallah S.** *Design and Control of Quadrotors*. 2007.
2. **Erginer B. Altug E.** *Modeling and PD Control of a Quadrotor VTOL Vehicle*. 2007.
3. **J. Brzozka.** *Regulatory i układy automatyki*. 2004.
4. **M. Gräber.** *Practical PID Tuning Guide*. 2021.
5. **Microelectronics STM.** Reference Manual STM32L4.
<https://www.st.com/en/microcontrollers-microprocessors/stm32l432kc.html>. [Online] 2023.
6. **Titterton D. Weston J.,.** *Strapdown Inertial Navigation Technology,*. 2004.
7. **JLCPCB.** [Online] <https://jlcpcb.com/>.
8. **Fly Sky.** iBUS Protocol Specification. [Online] <https://forum.arduino.cc/t/ibus-protocol-decoded/1073658>.
9. **Castillo P. Lozano R., Dzul A.** *Modelling and Control of Mini-Flying Machines*. 2005.
10. **Bosch Smart sensor: BNO055.** [Online] 2020. <https://www.bosch-sensortec.com/products/smart-sensor-systems/bno055/>.
11. **BLHeli_S.** BLHeli_S 4-in-1 ESC User Manual. [Online]
https://github.com/bitdump/BLHeli/blob/master/BLHeli_S%20SiLabs/BLHeli_S%20manual%20SiLabs%20Rev16.x.pdf.

SPIS RYSUNKÓW

Rysunek 1: Wizualizacja sposobu obrotu drona w osiach Roll, Pitch i Yaw	9
Rysunek 2: Wizualizacja rozmieszczenia silników i kierunków ich obrotu w konfiguracji "X"	11
Rysunek 3: Wizualizacja sposobu kontroli osi Roll, Pitch, Yaw	11
Rysunek 4: Wizualizacja ułożenia osi w wybranych czujnikach inercyjnych	12
Rysunek 5: Schemat działania regulatora PID	14
Rysunek 6: Dedykowana, autorska płyta PCB do sterowania dronem	20
Rysunek 7: Diagram połączeń elektronicznych	21
Rysunek 8: Konfiguracja Pinów mikrokontrolera STM32L432KC w programie STM32CubeMX.....	27
Rysunek 9: Schemat blokowy pętli sterowania.....	28
Rysunek 10: Osi przechyłu drona i ich przypisanie do pilota RC.....	32
Rysunek 11: Odczyt i modyfikacja zmiennych live w debugerze STM Cube IDE	40
Rysunek 12: Testy Drona - start z pozycji maksymalnie wychylonej.....	41
Rysunek 13: Obserwacje wychyleń osi ROLL i PITCH w aplikacji PuTTY podczas wychylenia maksymalnego drona na stanowisku testowym.....	42
Rysunek 14: Wpływ poszczególnych członów regulatora PID na jego wyjście.....	43
Rysunek 15: Dron podczas pozycji (0,0) w trakcie działania z poprawnie dobranymi parametrami PID	45